

Join the lecture online on your dashboard.

Scaling Self Attention: Controlling for variance

The Architecture Discrepancy

When transitioning from first-principles theory to the actual implementation in the original Transformer architecture, a deliberate mathematical modification appears.

First-Principles Theory

$$\text{softmax}(\mathbf{Q} \mathbf{K}^T) V$$

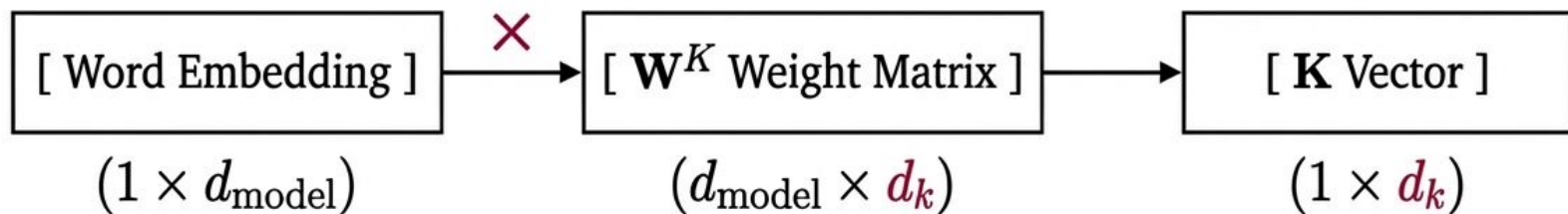
Original Implementation (Vaswani et al., 2017)

$$\text{softmax}\left(\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{d_k}}\right) V$$

The Core Question: Why must we scale the dot product by $1/\text{sqrt}(d_k)$?
Without this precise division, the model experiences severe training instability.

Defining the Variable: What is d_k ?

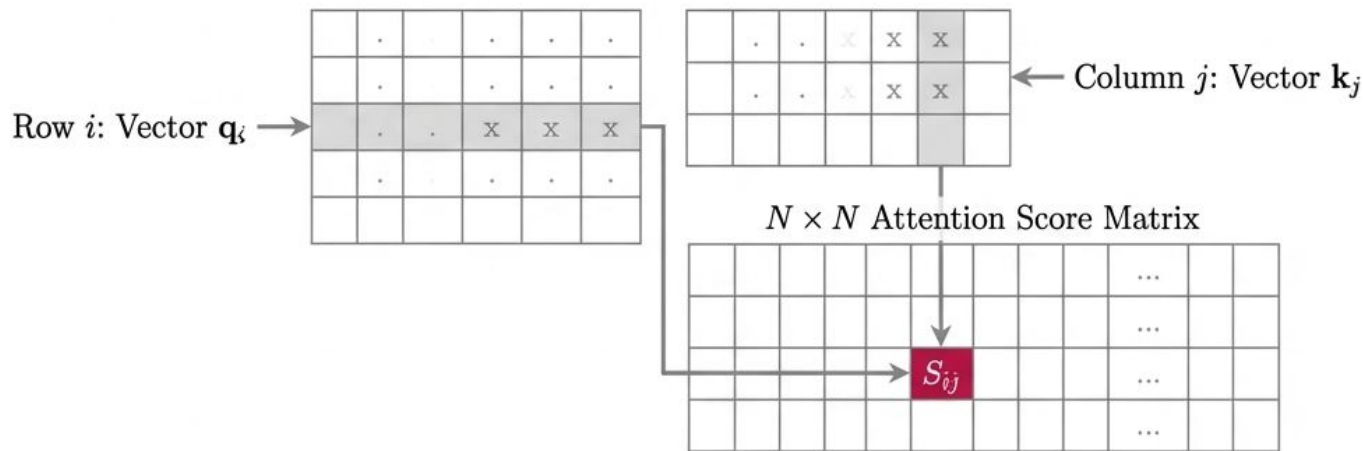
d_k represents the dimensionality of the Key (**K**) vectors (and, correspondingly, the Query vectors). It dictates the size of the arrays being multiplied.



- If a simplified Key vector has **3** numbers, $d_k = 3$.
- In standard large-scale Transformers, d_k is typically high-dimensional (e.g., $d_k = 512$).
- **Crucial Rule:** d_k is uniform across all tokens in the sequence ($\mathbf{K}_{\text{money}}$ and $\mathbf{K}_{\text{bank}}$ share the exact same d_k).

Anatomy of $Q K^T$: Behind the Matrix Multiplication

To understand the need for scaling, we must examine the fundamental operation generating the $Q K^T$ matrix: the vector-to-vector dot product.



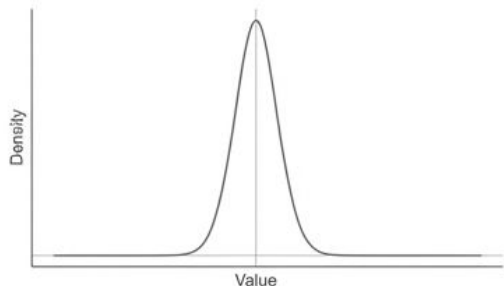
$$S_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j = \sum_{m=1}^{d_k} (\mathbf{q}_{i,m} \times \mathbf{k}_{j,m})$$

The final output matrix is simply a grid of individual scalar values, each generated by summing d_k independent multiplications.

The Core Hypothesis: Dimensionality Drives Variance

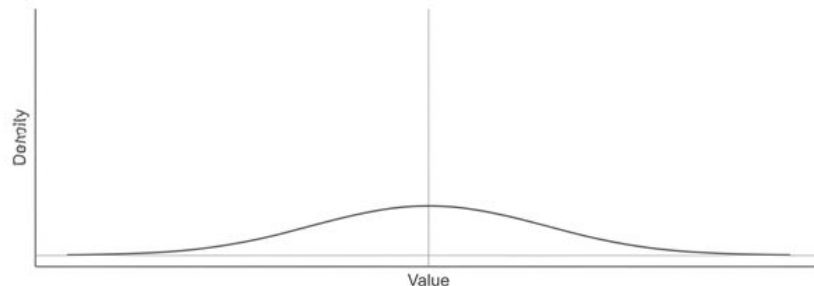
The arithmetic nature of the dot product dictates a direct, mathematically unavoidable relationship between vector dimensionality (d_k) and statistical variance.

Condition A: Low-Dimensional (e.g., $d_k = 3$)



- Mechanics: Summing only 3 scalar products.
- Output Profile: Highly concentrated, narrow distribution of values.
- Result: **Low Variance** within the $Q K^T$ matrix.

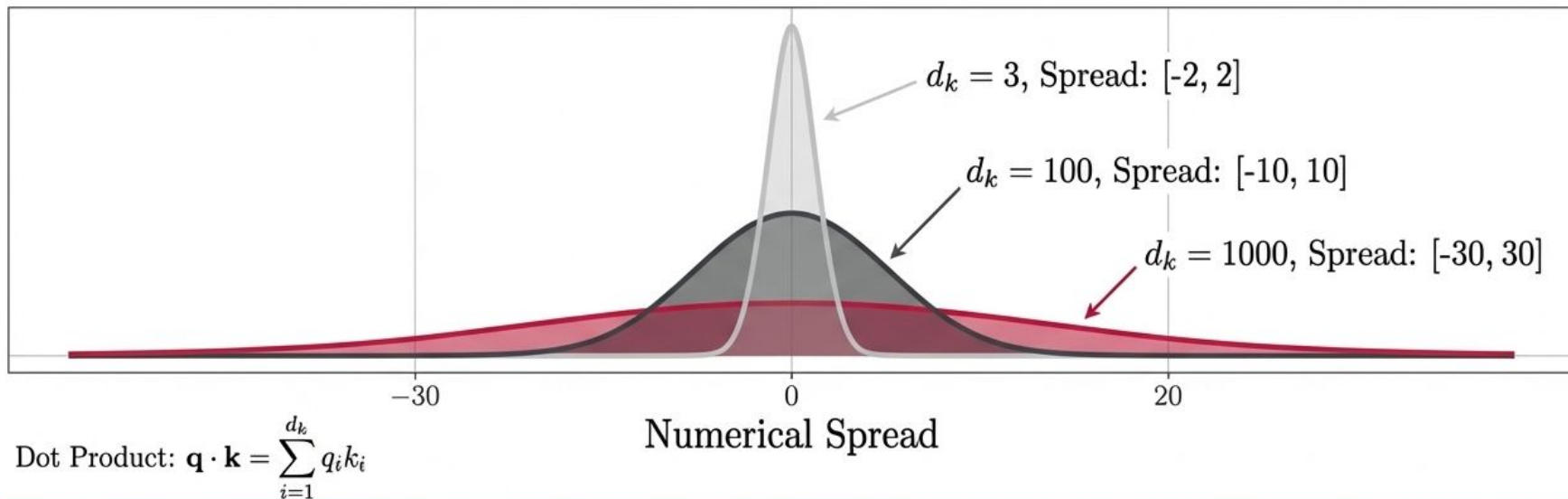
Condition B: High-Dimensional (e.g., $d_k = 512$)



- Mechanics: Summing 512 independent scalar products.
- Output Profile: Massive spread of exceptionally large positive and negative numbers.
- Result: **High Variance** within the $Q K^T$ matrix.

Empirical Evidence: The Variance Explosion

Computing the dot products of 1,000 randomly initialized vector pairs empirically proves that as dimensions increase, the variance expands aggressively.

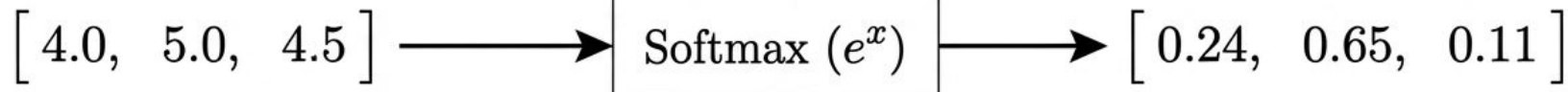


Higher d_k guarantees an extreme variety of numbers (from huge positives to huge negatives).
In neural networks, uncontrolled high variance is a fatal pathology.

The The Catalyst: Softmax Exponential Sensitivity

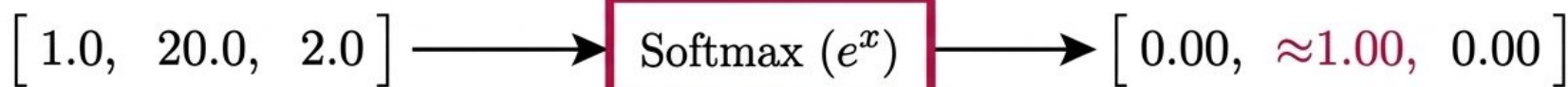
Raw dot products are immediately fed into a Softmax layer. Because Softmax relies on exponentiation, it is radically hypersensitive to the variance of its input array.

Healthy / Low Variance Input



Smooth, distributed probabilities. The model effectively distributes its attention.

Pathological / High Variance Input



Extreme polarization. Softmax aggressively forces the largest number to 100% and crushes everything else to absolute zero.

The Probability Collapse

When modern transformer architectures utilize the necessary high-dimensional embeddings, the statistical reality of the dot product fundamentally breaks the Softmax mechanism.

Step 1: High Dimensionality ($d_k \uparrow$) inherently generates High Variance in the raw QK^T matrix.



Step 2: High variance introduces massive outliers into the exponential Softmax Layer.



Step 3: Softmax outputs **Extreme Probabilities** (polarizing to ≈ 1 for the maximum value, and ≈ 0 for all others).

The resulting question: Why are polarized probabilities mathematically dangerous during network training?

Formulating the Engineering Objective

We cannot simply reduce dimensionality (d_k), as large embeddings are strictly required to capture complex semantic meaning. Instead, we must mathematically **compress** the variance.

Goal: Apply a mathematical transformation to the $\mathbf{Q} \mathbf{K}^T$ matrix that artificially forces its variance to remain constant—matching a low-dimensional baseline—**regardless** of how large d_k grows.

The Mechanism: Divide the entire matrix by an unknown constant scaling factor (c).

$$\text{Target Matrix} = \frac{\mathbf{Q} \mathbf{K}^T}{c}$$

The Challenge: What exact mathematical value must c be to perfectly neutralize the variance added by d_k ?

Mathematical Proof: Scaling to d_k Dimensions

As vectors expand to multiple dimensions, the total dot product becomes the sum of independent 1D dot products.

For a d_k -Dimensional vector:

$$S = q \cdot k = \sum_{i=1}^{d_k} (q_i \times k_i)$$

$$S = x_1 + x_2 + x_3 + \dots + x_{d_k}$$

Calculating the Total Variance: Because the random variables are independent, the variance of the sum is exactly the sum of the variances:

$$\text{Var}(S) = \text{Var}(x_1) + \text{Var}(x_2) + \dots + \text{Var}(x_{d_k})$$

$$\text{Var}(S) = \mathbf{d_k} \times \text{Var}(x)$$

Theorem Statement: Total variance grows linearly with dimension d_k .

Deriving the Perfect Scaling Factor

We construct an equation forcing the variance of our d_k -dimensional dot product back to the baseline 1D variance, solving for c .

Step 1 (The Setup): Let our scaling constant be c .

$$\text{Var}(c \cdot S) = \text{Baseline Variance}$$

Step 2 (Apply Theorem & Substitute):

$$c^2 \cdot \text{Var}(S) = \text{Var}(x)$$

$$c^2 \cdot (d_k \cdot \text{Var}(x)) = \text{Var}(x)$$

Step 3 (Isolate c): Cancel $\text{Var}(x)$ from both sides.

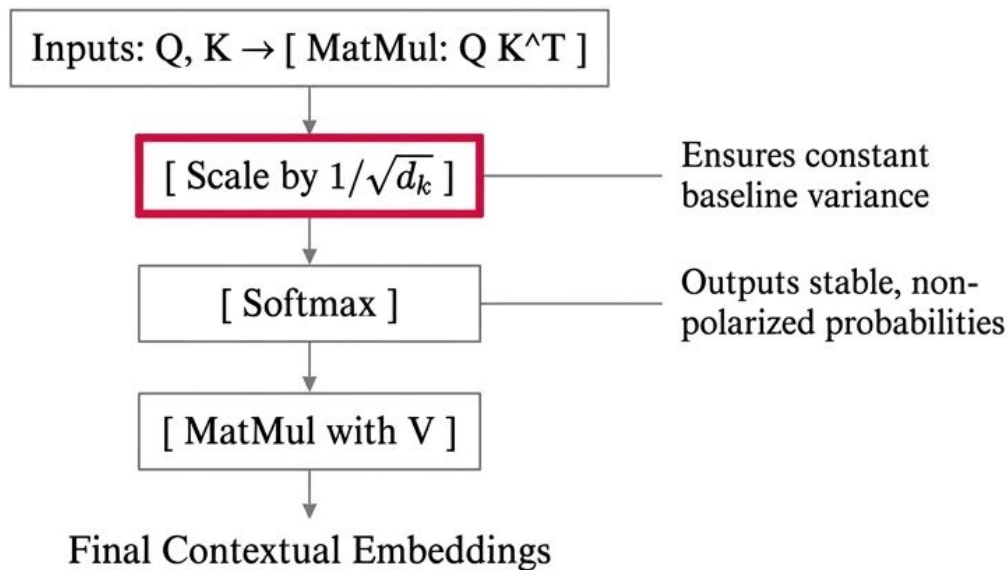
$$c^2 \cdot d_k = 1$$

$$c^2 = 1/d_k$$

$$c = \frac{1}{\sqrt{d_k}}$$

The Final Architecture: Scaled Dot-Product Attention

By scaling the raw attention scores by $1/\sqrt{d_k}$, we guarantee a stable, low variance regardless of embedding size. This ensures healthy Softmax probability distribution and robust gradient flow.



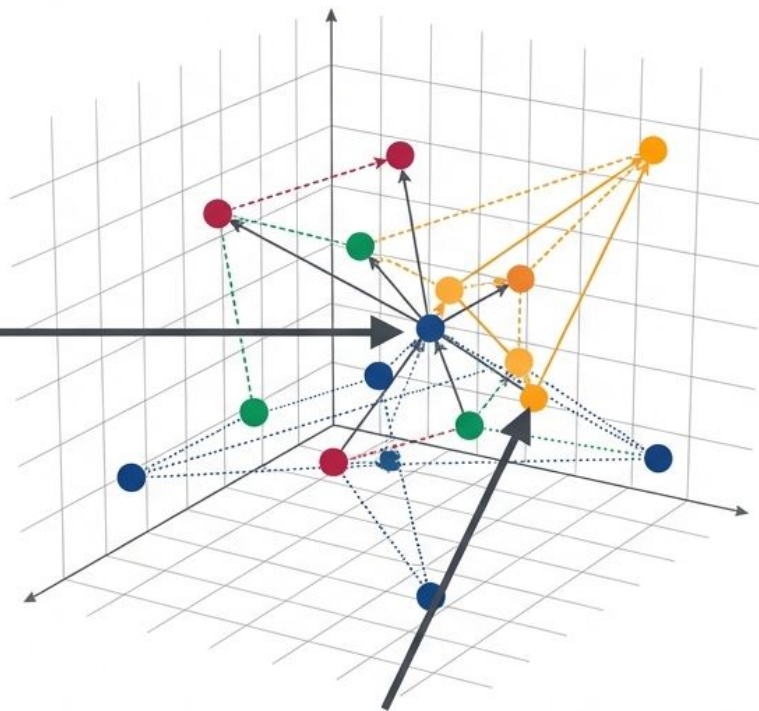
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Geometry of Self Attention

Translating Algebraic Formulas into Geometric Mechanics

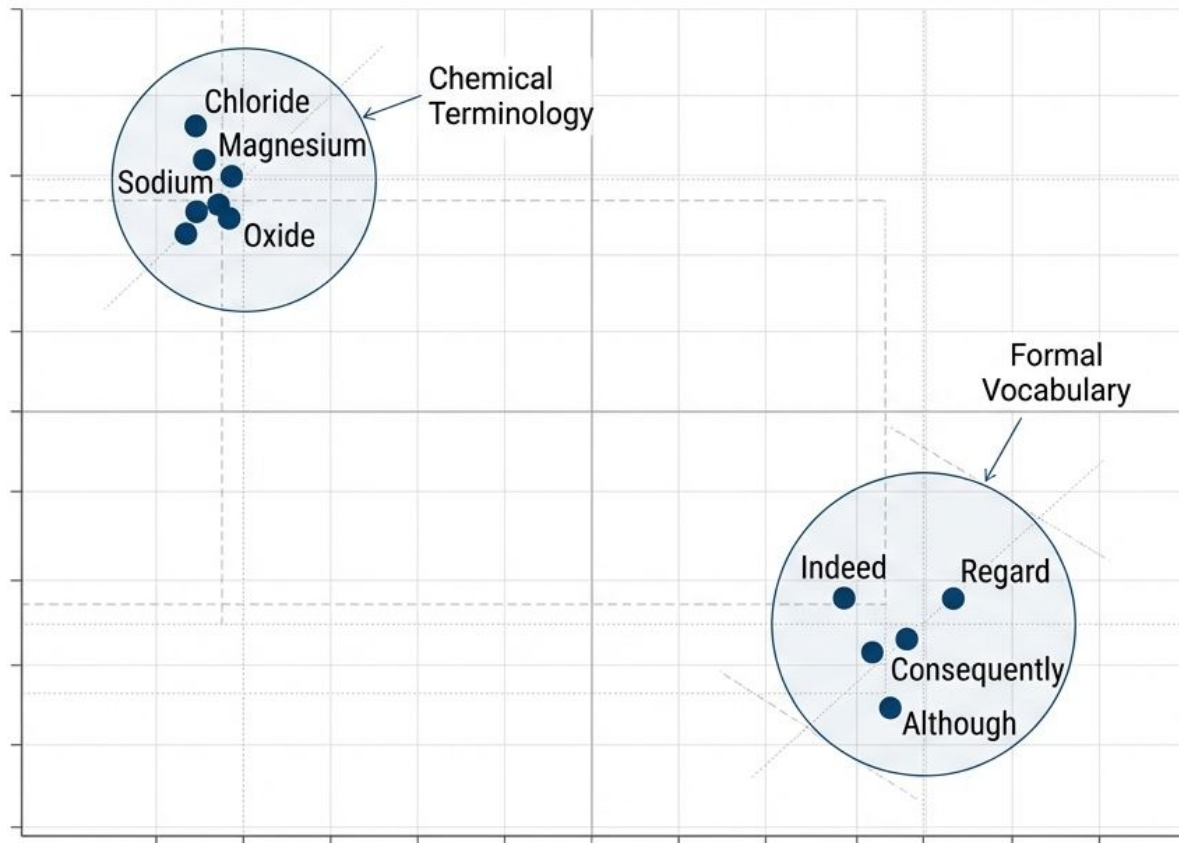
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

We know the sequence of operations. Now, we decode what happens to a word's physical position in space during this computation.



Mathematical operations in self-attention are instructions for geometric movement.

Projecting Semantic Meaning onto a 2D Cartesian Canvas

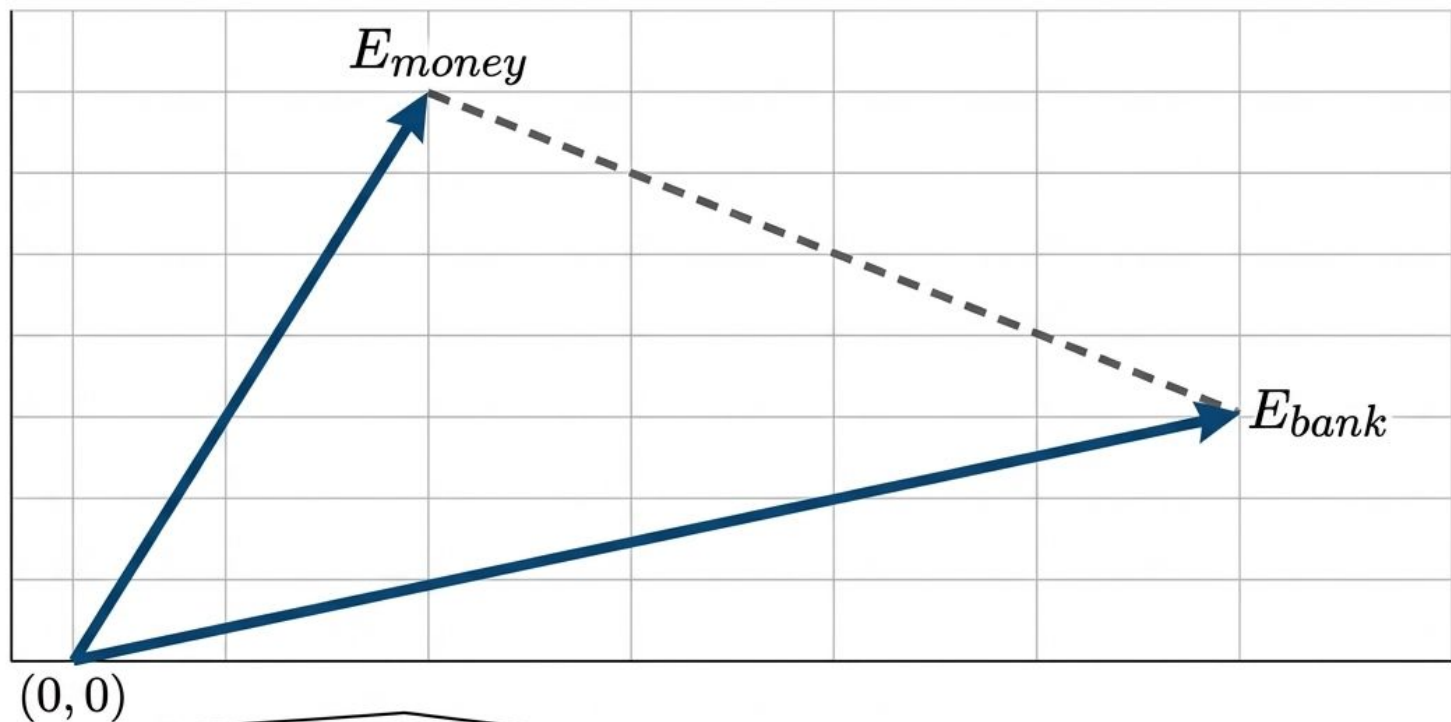


Word embeddings typically exist in high-dimensional spaces (e.g., 200D).

To visualize the mechanics of self-attention, we project them into a comprehensible 2-Dimensional Cartesian space.

Key Rule: Words are physical coordinates. Proximity equals semantic relationship.

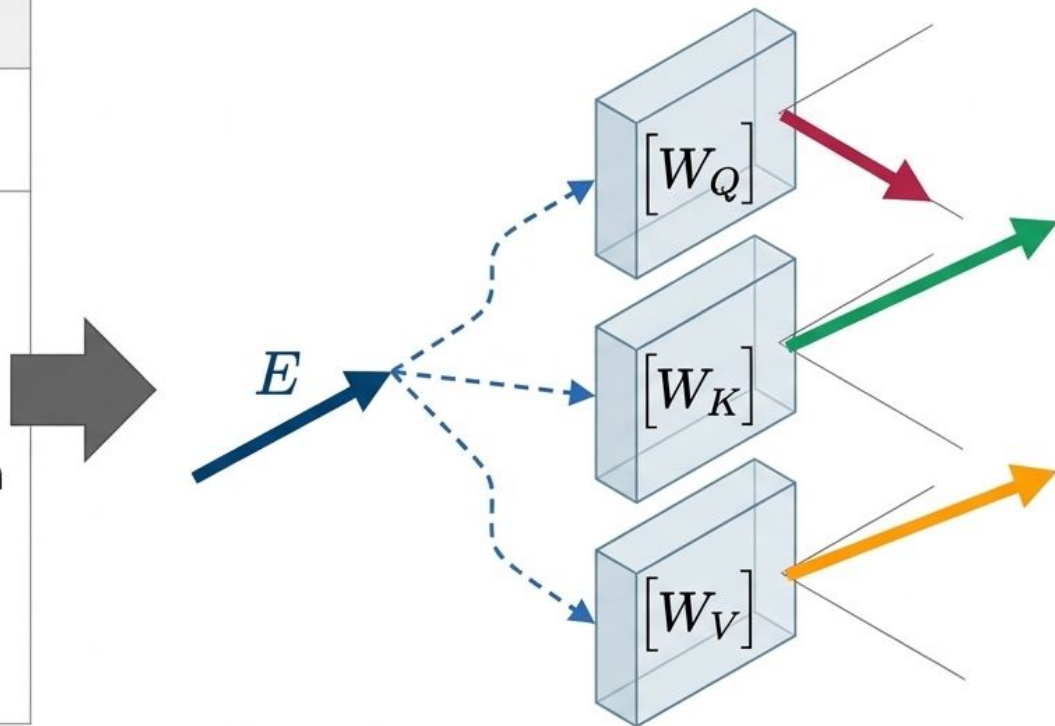
The Initial State: Isolated Word Embeddings



In an isolated state—derived from standard Word2Vec before any attention is applied—‘Money’ and ‘Bank’ exist at fixed, distant coordinates with zero contextual influence on each other.

Weight Matrices as Engines of Spatial Transformation

1	2
Mathematical Operation	Geometric Equivalent
Matrix Multiplication ($E \times W$)	Linear Transformation



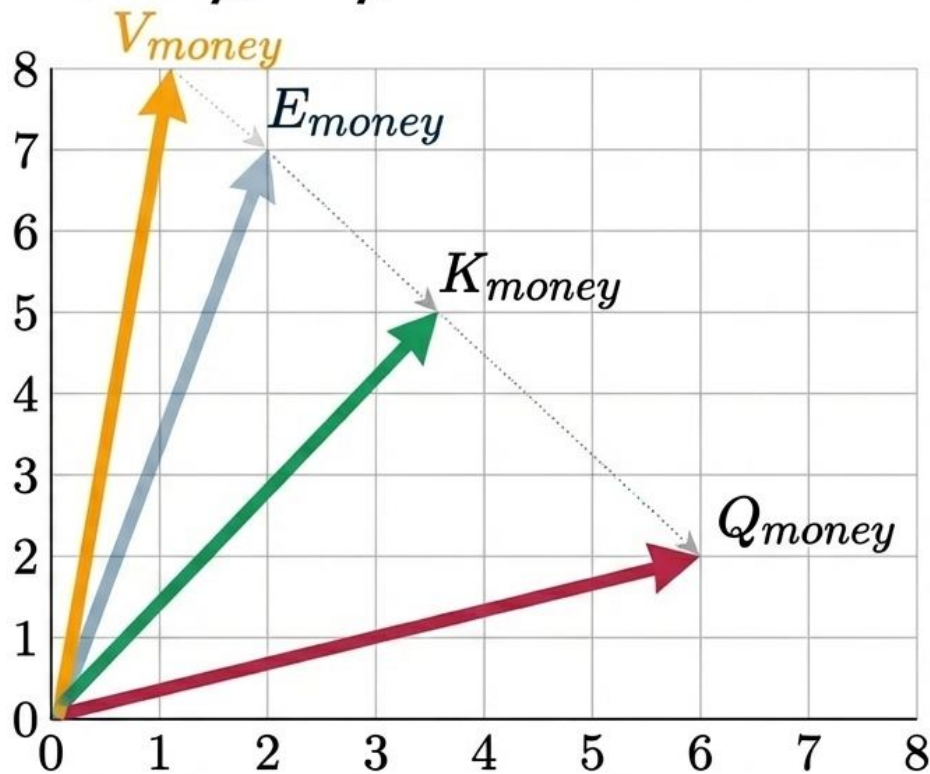
Multiplying our 2D embedding by a 2×2 weight matrix is not just number crunching—it is a physical rotation and scaling of the original vector to a new role-specific position.

Splitting an Embedding into Query, Key, and Value States

$$E_{\text{money}} \times W_Q = Q_{\text{money}}$$

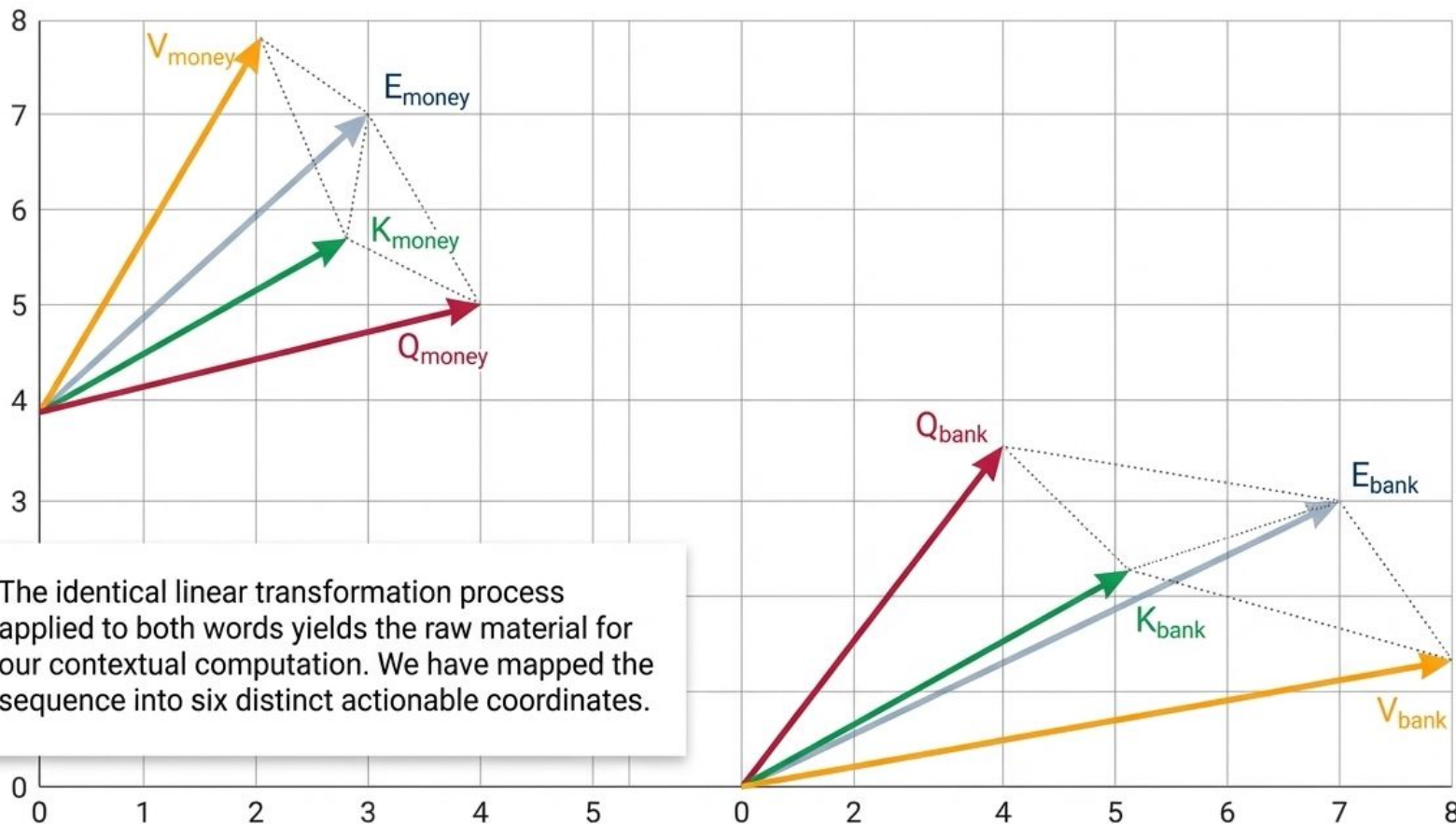
$$E_{\text{money}} \times W_K = K_{\text{money}}$$

$$E_{\text{money}} \times W_V = V_{\text{money}}$$

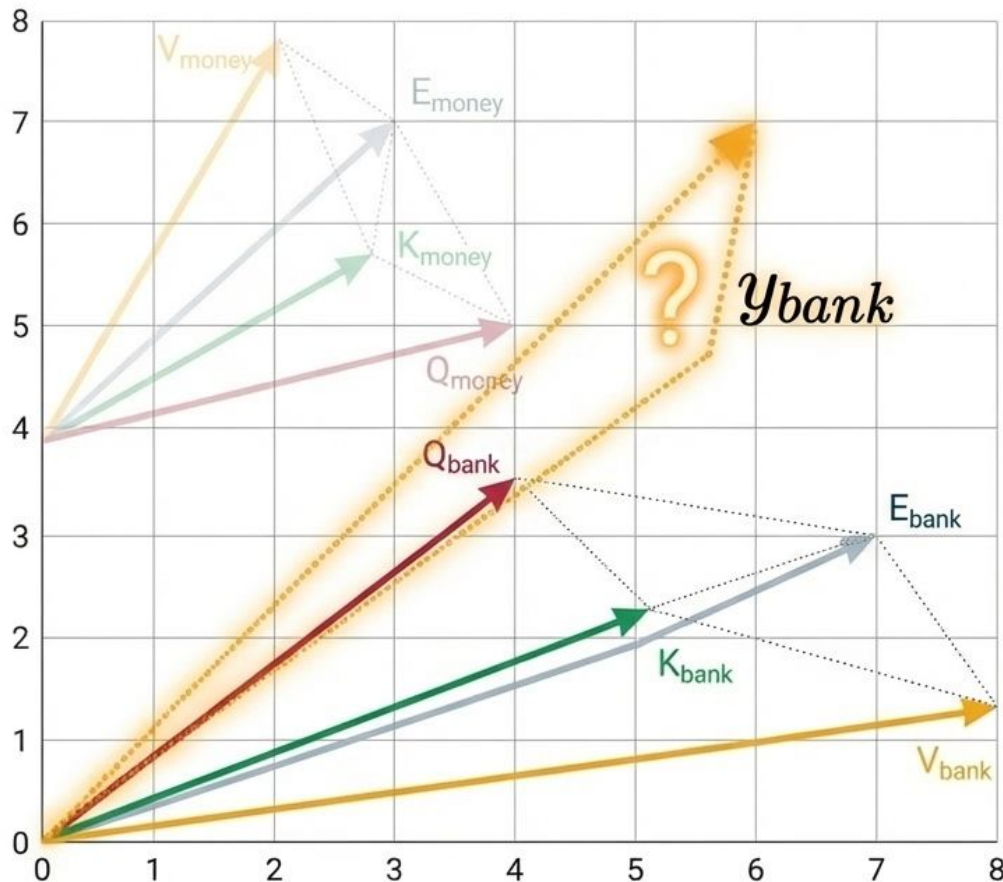


A single static representation is fractured into three dynamic geometric roles:
a Query (what I seek), a Key (what I contain), and a Value (what I offer).

The Complete 6-Vector Operational State Space



Defining the Objective: Discovering the Contextual Target



The Mission

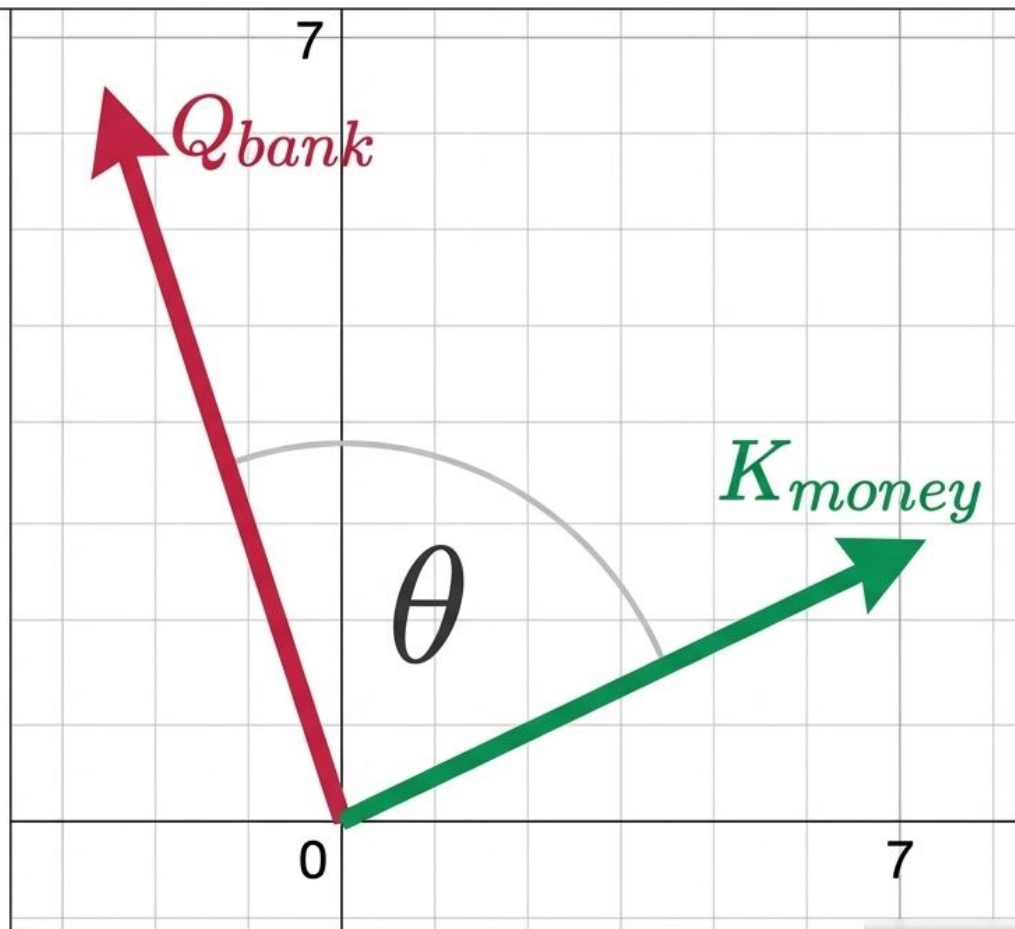
- Calculate y_{bank} — the context-aware embedding of the word "Bank" given its sequential proximity to "Money".
- We are shifting "Bank" from an isolated dictionary definition to a dynamic, localized reality.

Evaluating Cross-Alignment: The Geometry of Dot Products

$$Q_{bank} \cdot K_{money} = \text{Raw Score}$$

$$\text{Score} = 10$$

The dot product measures angular distance. A large geometric angle between a Query and a Key signifies a Query and a Key signifies low semantic alignment, yielding a minimal mathematical similarity score (e.g., 10).

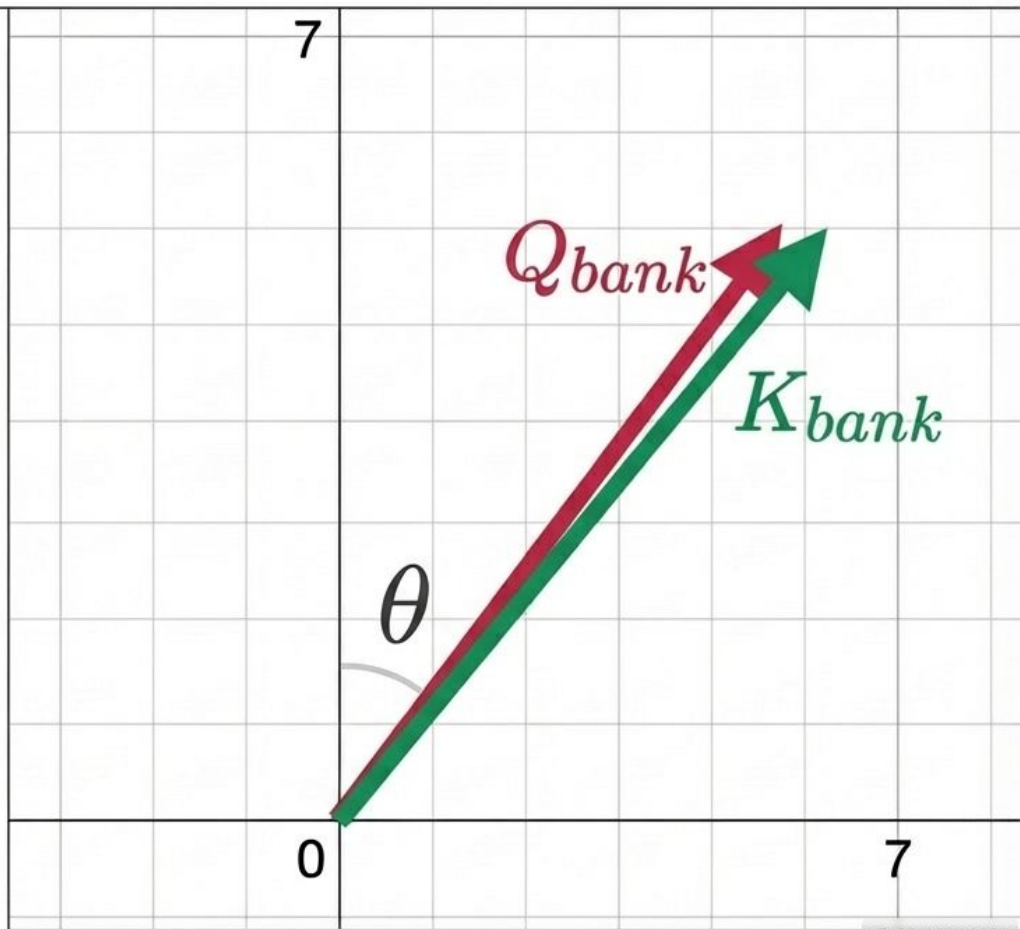


Evaluating Self-Alignment: Native Context is Geometrically Tight

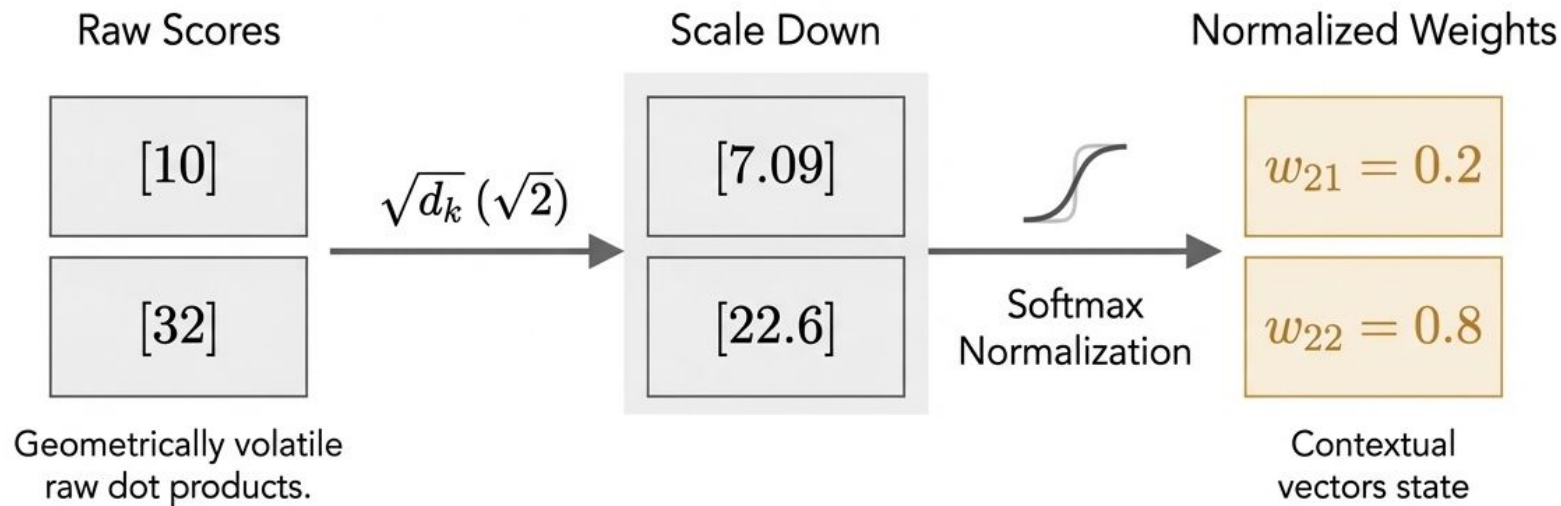
$$Q_{bank} \cdot K_{bank} = \text{Raw Score}$$

$$\text{Score} = 32$$

Comparing the word to its own native Key. The extremely tight angular distance results in a proportionally massive dot product (e.g., 32). Words naturally exhibit the strongest geometric alignment with themselves.



Stabilizing Geometry into Actionable Probability Weights



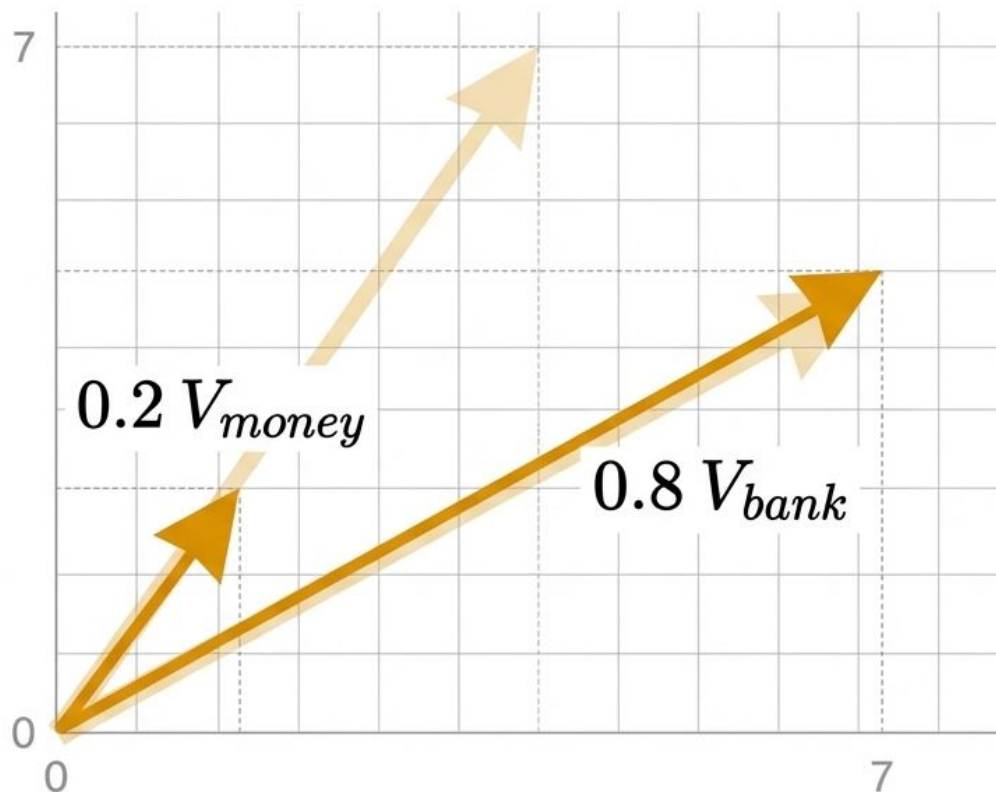
Softmax is a regulatory mechanism. It forces abstract, unbounded spatial alignments to compete, resulting in fractional semantic weights that cleanly sum to 1.

Applying Softmax Weights to Value Vectors

$$0.2 \times V_{money}$$

$$0.8 \times V_{bank}$$

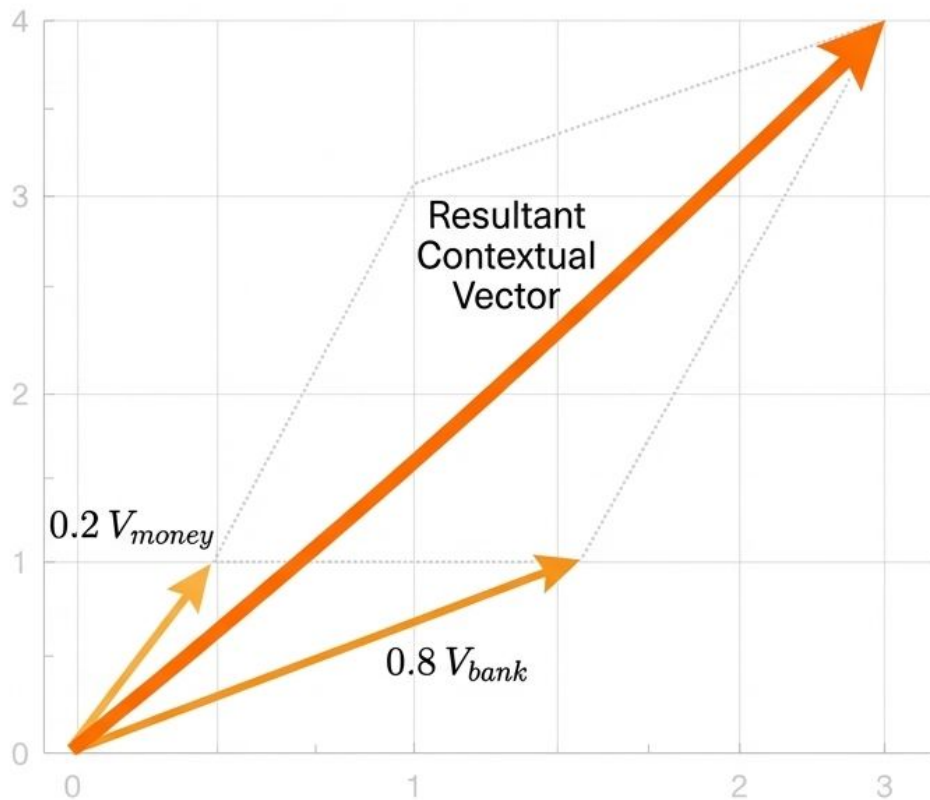
Scalar multiplication physically stretches or squashes the semantic "Value" of a word based entirely on its calculated relevance.



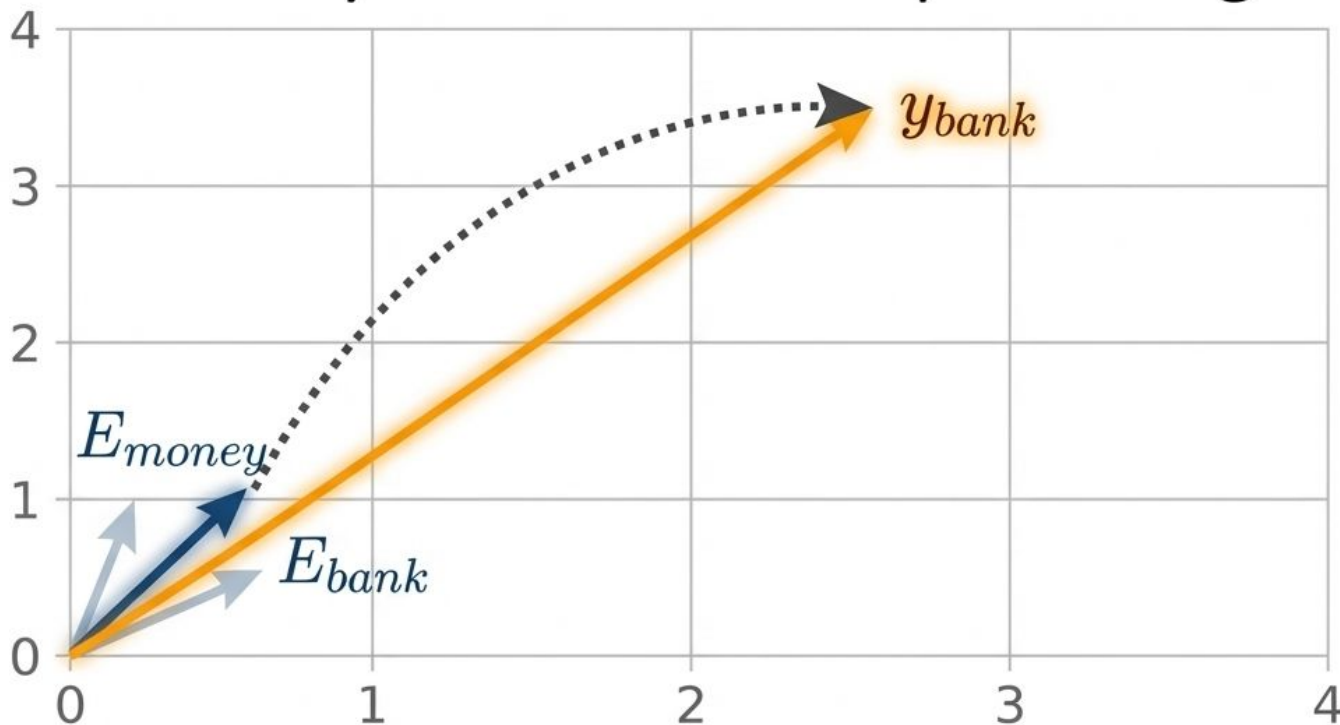
Synthesizing Context via Vector Addition

Mathematical Operation	Geometric Equivalent
Addition ($\sum W \cdot V$)	Parallelogram Law

To combine the contextual influences, we add the scaled vectors. In our spatial canvas, this executes flawlessly using the Parallelogram Law of vector addition.

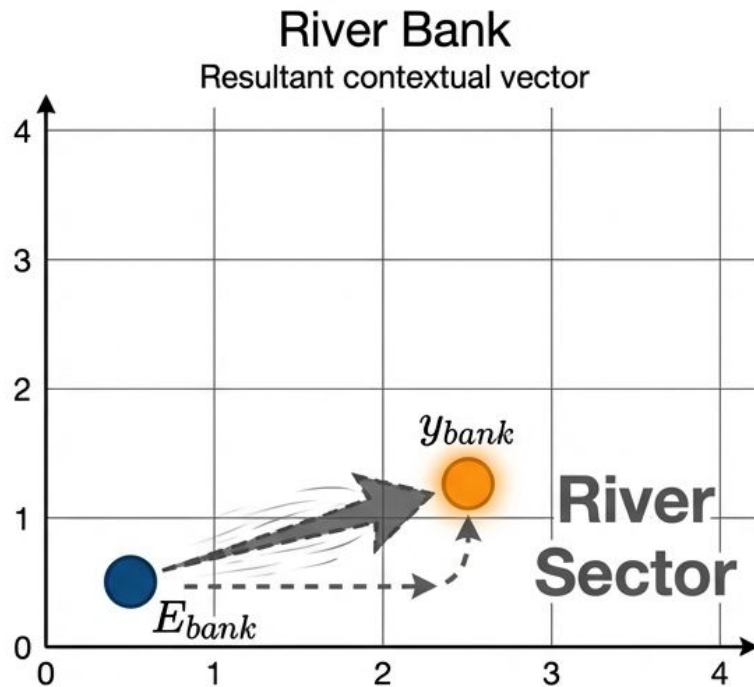
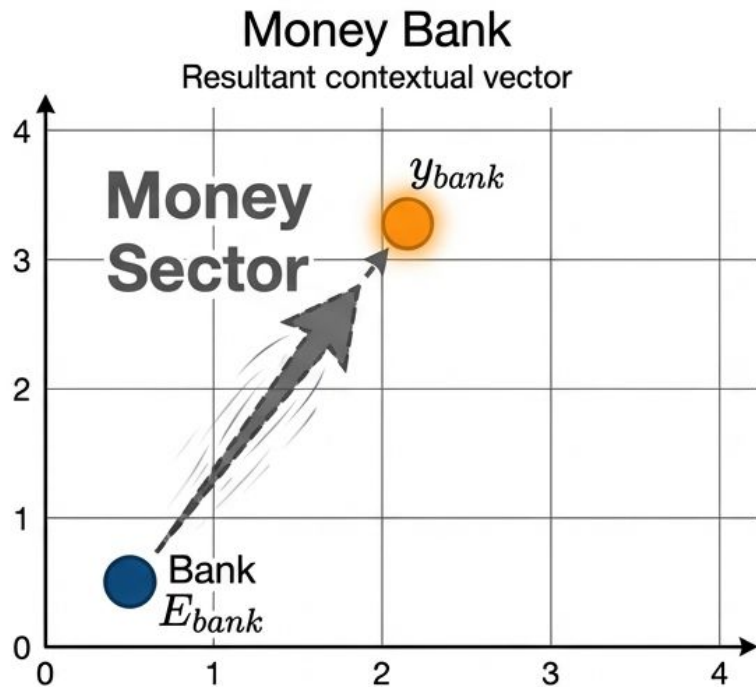


The Geometry of Context: A Spatial Migration



The brilliance of Self-Attention revealed: The representation of 'Bank' literally traveled through space. It abandoned its generic, isolated dictionary position and migrated closer to 'Money'.

Self-Attention as Semantic Gravity

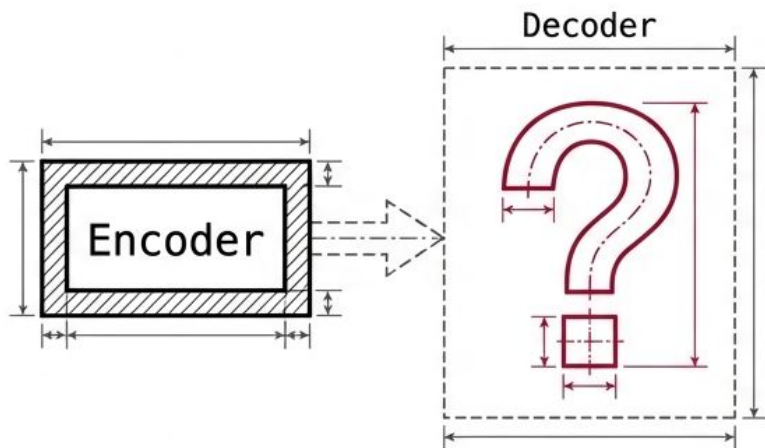


Self-Attention is the **physics engine** of **language models**. Words exert a proportional gravitational pull on one another based on learned training patterns. The final position position of any word is dynamically dictated by the semantic gravity of its neighbors.

The “Self” in Self Attention

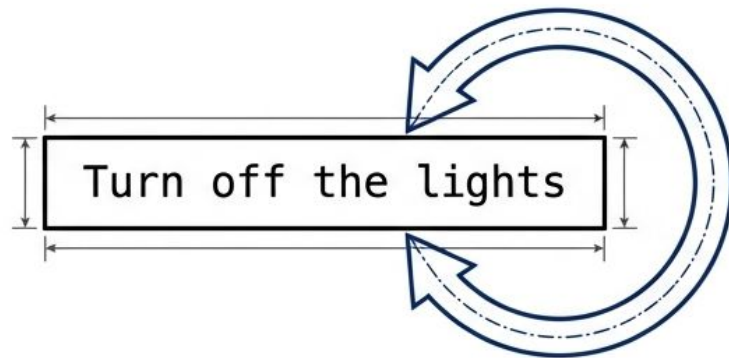
The Two Paradoxes of Self-Attention

Paradox 1: How is this an Attention mechanism?



Classical attention connects an Encoder to a Decoder. Self-Attention has neither. How can the mathematical formulation survive?

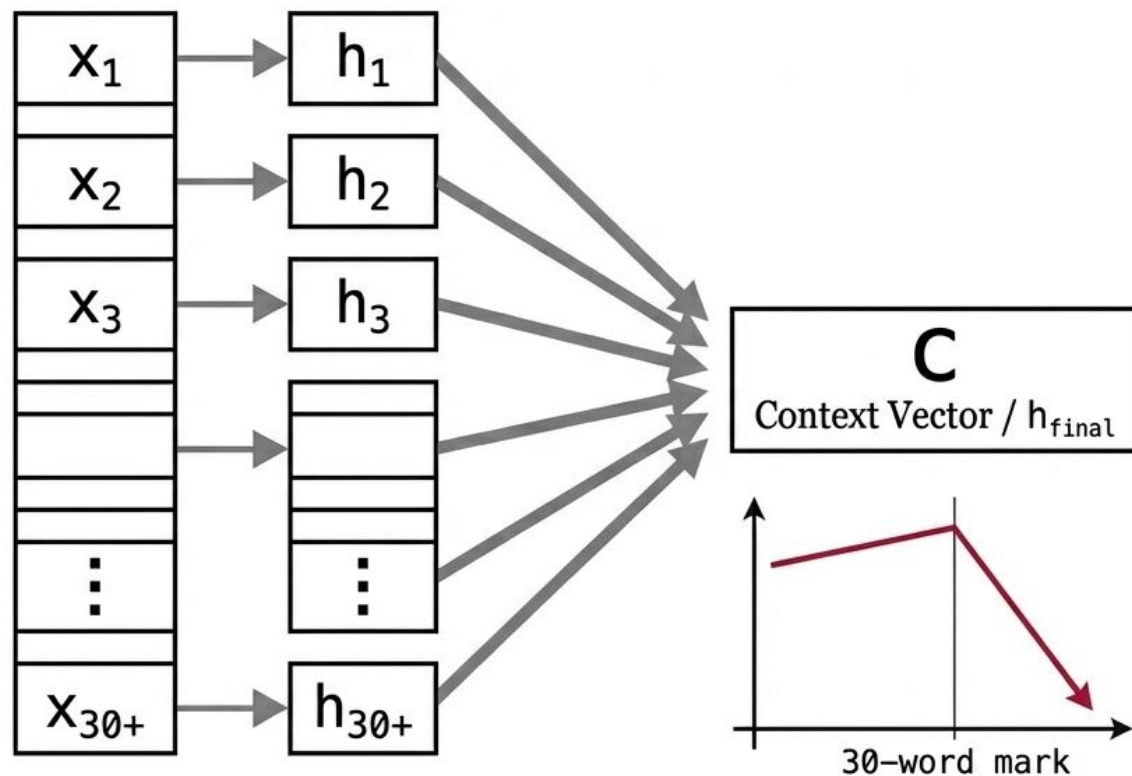
Paradox 2: Why is it called *Self*?



If there is only one sequence, who is paying attention to whom?

To answer the second question, we must mathematically prove the first.

The Context Vector Bottleneck



The System:

Traditional Seq2Seq (Encoder-Decoder without Attention).

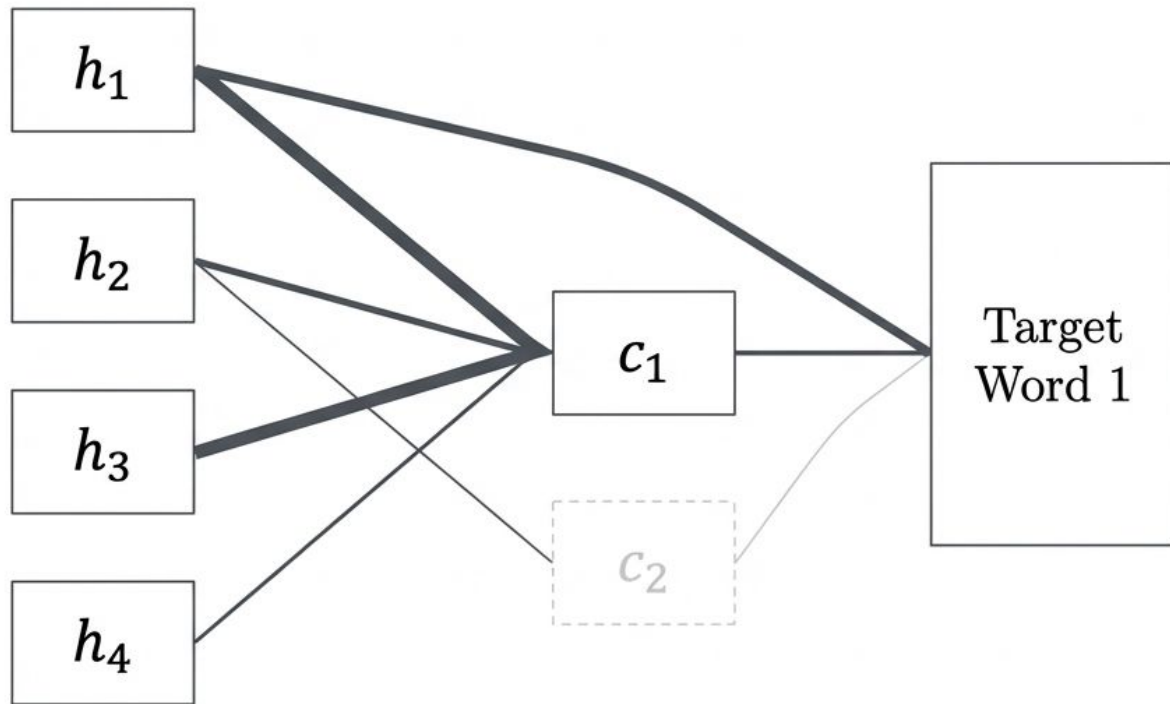
The Flaw:

Delegating the comprehension of an entire complex sequence to a single, fixed-length set of numbers.

The Result:

Degradation of translation quality. A static context vector cannot retain complete sequential information for sentences exceeding 30 words.

Baseline Recap: The Luong Attention Paradigm



The Paradigm Shift:

Instead of one bottleneck summary, create a bespoke context vector for every single output time-step.

Mechanism:

When predicting the first word ("Light"), the system generates a unique context vector (c_1) containing the exact weights of which input words are most useful.

Result:

The system load is distributed. The decoder looks at the entire input sequence dynamically at each step.

The Mathematics of Luong Alignment

1. Alignment Score

$$\mathbf{e}_{ij} = \mathbf{s}_i \cdot \mathbf{h}_j$$

(Decoder hidden state)

(Encoder hidden state)

2. Attention Weights

$$\alpha_{ij} = \text{Softmax}(\mathbf{e}_{ij})$$

Normalizes scores into probabilities.

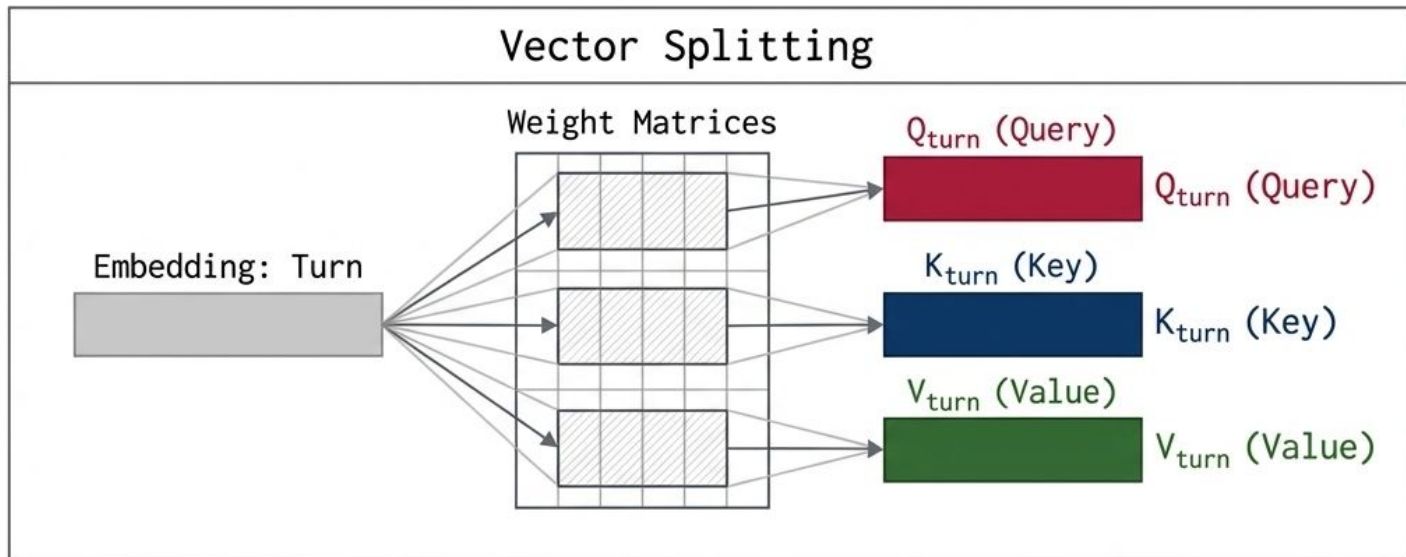
3. Context Vector

$$\mathbf{c}_i = \sum \alpha_{ij} \mathbf{h}_j$$

The weighted sum of encoder hidden states.

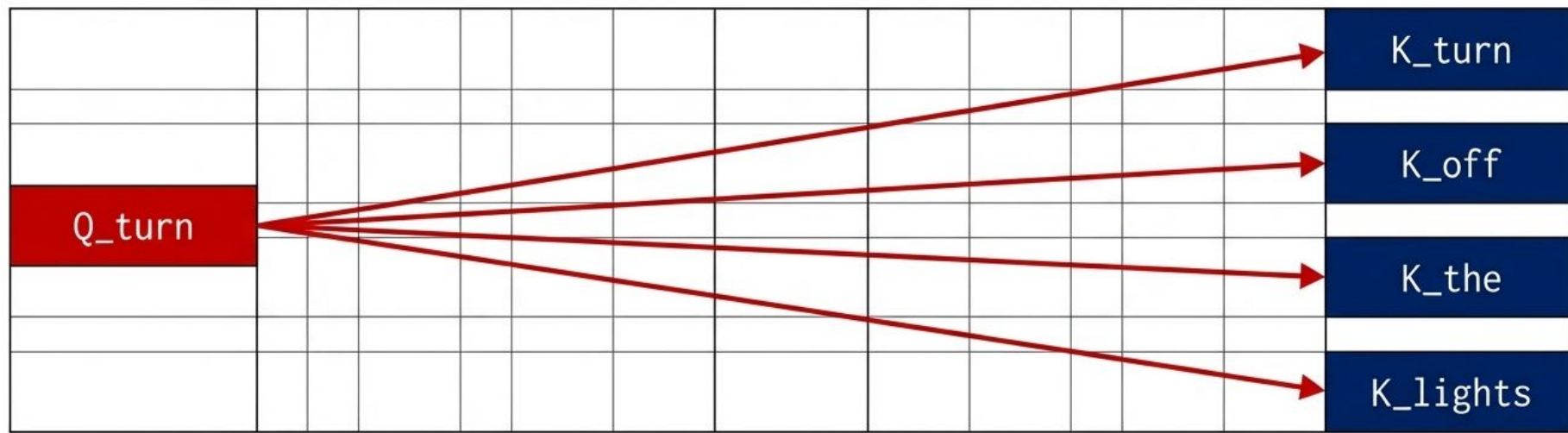
Core Principle: These three exact mathematical operations—Dot Product, Softmax, Weighted Sum—are the absolute definition of an Attention Mechanism.

Deriving the Q, K, V Triad



- **The Triad:** To simulate the asker/answerer dynamic internally, every single word in the sequence is projected into three distinct representations.
- **Rule of Scale:** This projection happens for every word. The sequence Turn off the lights yields 4 Queries, 4 Keys, and 4 Values.

The Intra-Sequence Query (Time-Step 1)



The Objective: Generate the final contextual embedding (Y_{turn}) for the first word.

The Cognitive Framing: The word Turn acts as the Decoder. It uses its Query vector to ask the rest of the sentence: Which of you is most useful for understanding my context?

The Targets: The sentence uses its Key vectors to answer the query.

Scoring Similarity in Self-Attention

$$Q_{\text{turn}} \cdot K_{\text{turn}} \rightarrow S_{11}$$

$$Q_{\text{turn}} \cdot K_{\text{off}} \rightarrow S_{12}$$

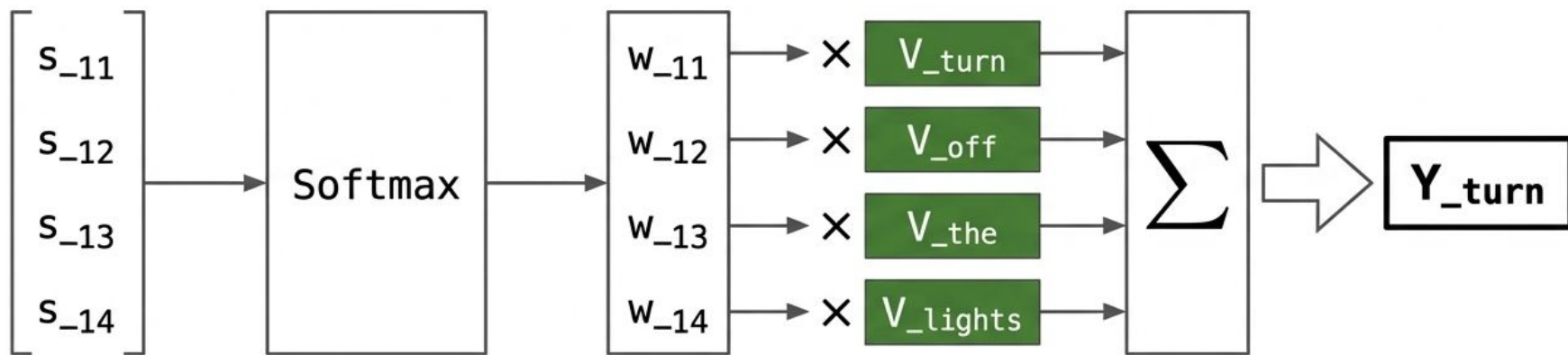
$$Q_{\text{turn}} \cdot K_{\text{the}} \rightarrow S_{13}$$

$$Q_{\text{turn}} \cdot K_{\text{lights}} \rightarrow S_{14}$$

The Math: Similarity is calculated via Dot Product ($Q \cdot K$).

The Output: This generates raw alignment scores (s_{11} through s_{14}), representing how much focus 'Turn' should place on every other word in the sequence.

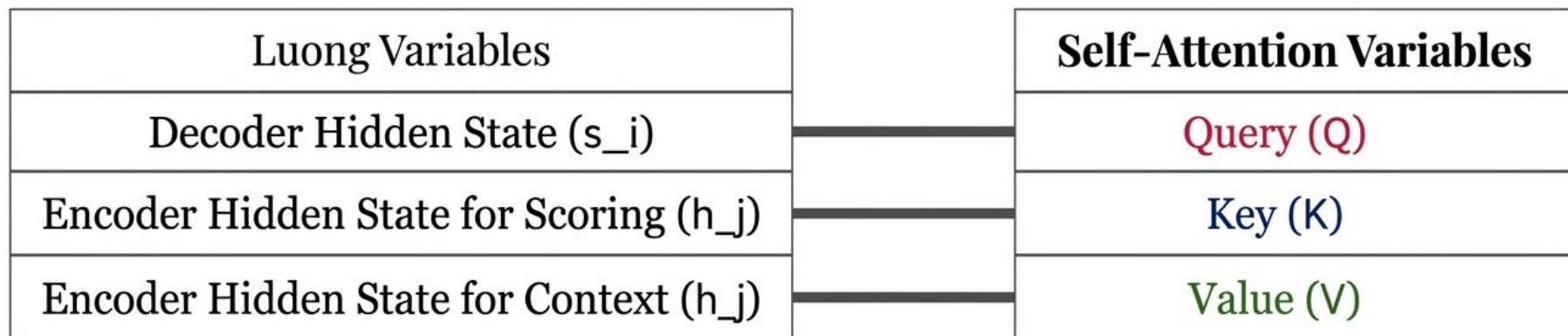
Normalization & The V-Weighted Sum



Final Equation: $\mathbf{Y}_{\text{turn}} = (w_{11} \cdot \mathbf{V}_{\text{turn}}) + (w_{12} \cdot \mathbf{V}_{\text{off}}) + (w_{13} \cdot \mathbf{V}_{\text{the}}) + (w_{14} \cdot \mathbf{V}_{\text{lights}})$

Result: $\mathbf{Y}_{\text{turn}}$ is the final, context-aware embedding for the first word. The system then repeats this exact loop for “off”, “the”, and “lights”.

Synthesis: The Conceptual Bridge



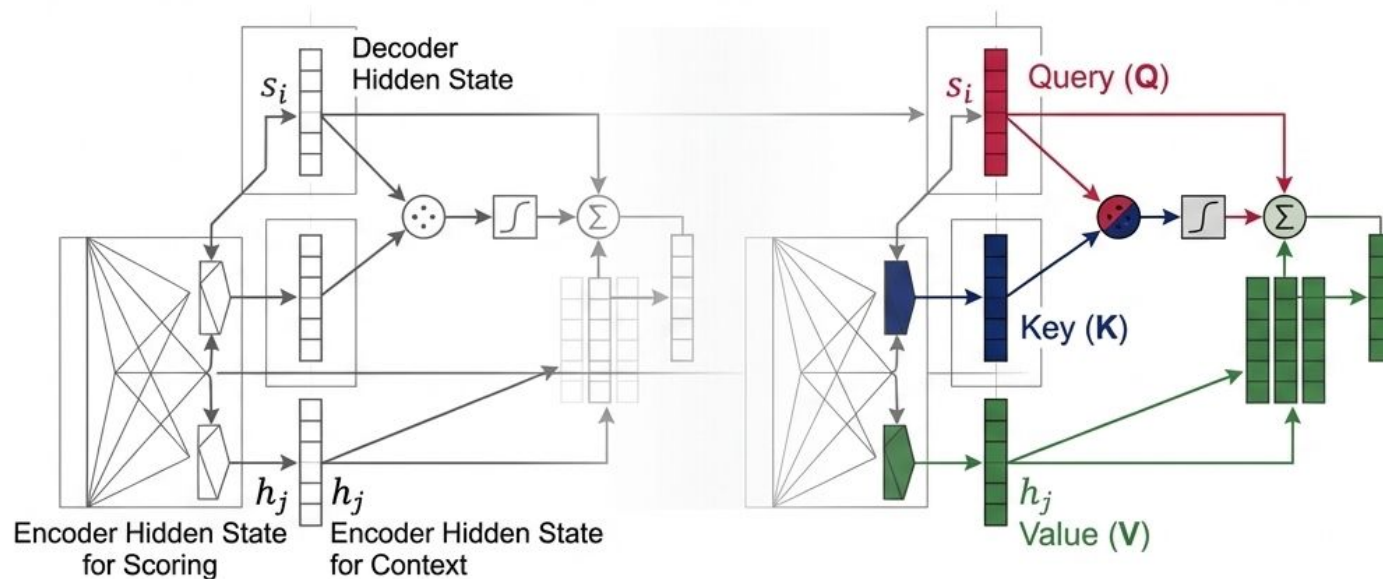
The Bridge: The distinct roles of the asker and the answerer still exist.
In Self-Attention, they are simply derived from the exact same input source rather than two separate neural networks.

Proof of Mathematical Equivalence

	Luong Equations	Self-Attention Equations
Alignment Score	$e_{ij} = s_i \cdot h_j$	$s_{ij} = \textcolor{red}{Q}_i \cdot \textcolor{blue}{K}_j$
Weights	$\alpha_{ij} = \text{Softmax}(e_{ij})$	$w_{ij} = \text{Softmax}(s_{ij})$
Context Vector	$c_i = \sum \alpha_{ij} h_j$	$Y_i = \sum w_{ij} \textcolor{green}{V}_j$

Diagnostic Conclusion: The equations are mathematically identical. The architecture changed, but the fundamental operations remained untouched.

Resolution 1: Why it is Attention

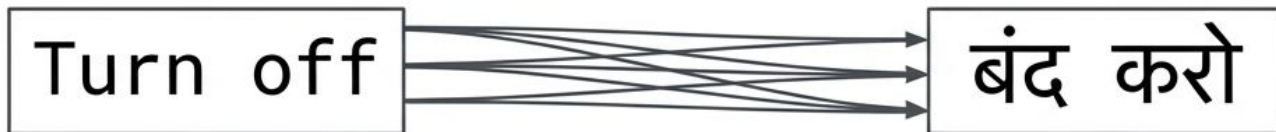


The Final Answer: Self-Attention is unequivocally an Attention Mechanism because the three pillars of calculation—dot product alignment, softmax normalization, and weighted summation—are preserved entirely.

The Insight: We are reusing the exact same equations in a different structural setting.

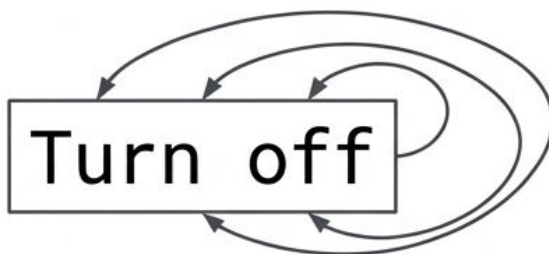
The Inter vs. Intra Paradigm

Classical Attention



Translation (Inter): Traditional attention acts as a bridge between two distinct, separate datasets.

Self-Attention



Comprehension (Intra): Self-Attention calculates alignment scores entirely within the boundaries of a single, identical sequence.

Resolution 2: The Meaning of Self

The Final Answer:

We call it Self because the sequence is quite literally literally paying attention to itself. The mathematical mechanisms of understanding are directed completely inward.



Module Conclusion:

Module Conclusion:

You have now mastered the mathematical blueprint of the Q-K-V intra-sequence logic.

The axe is sharp. Next Module -> Multi-Head Attention & Transformer Architecture.

Multi-Head Attention

Recap: The Core Function of Self-Attention

Static Embeddings (e.g., Word2Vec)



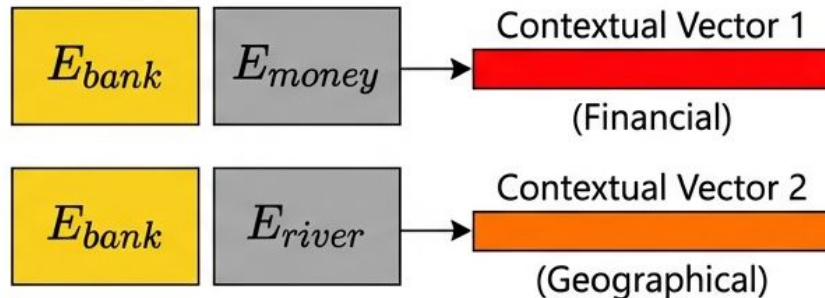
Mechanism: One fixed vector per word, regardless of context.

The Problem: The word Bank has the exact same numerical representation in:

- I need to withdraw money from the bank.
- We sat by the river bank.

Result: Fails to capture polysemy (multiple meanings).

Contextual Embeddings (Self-Attention)

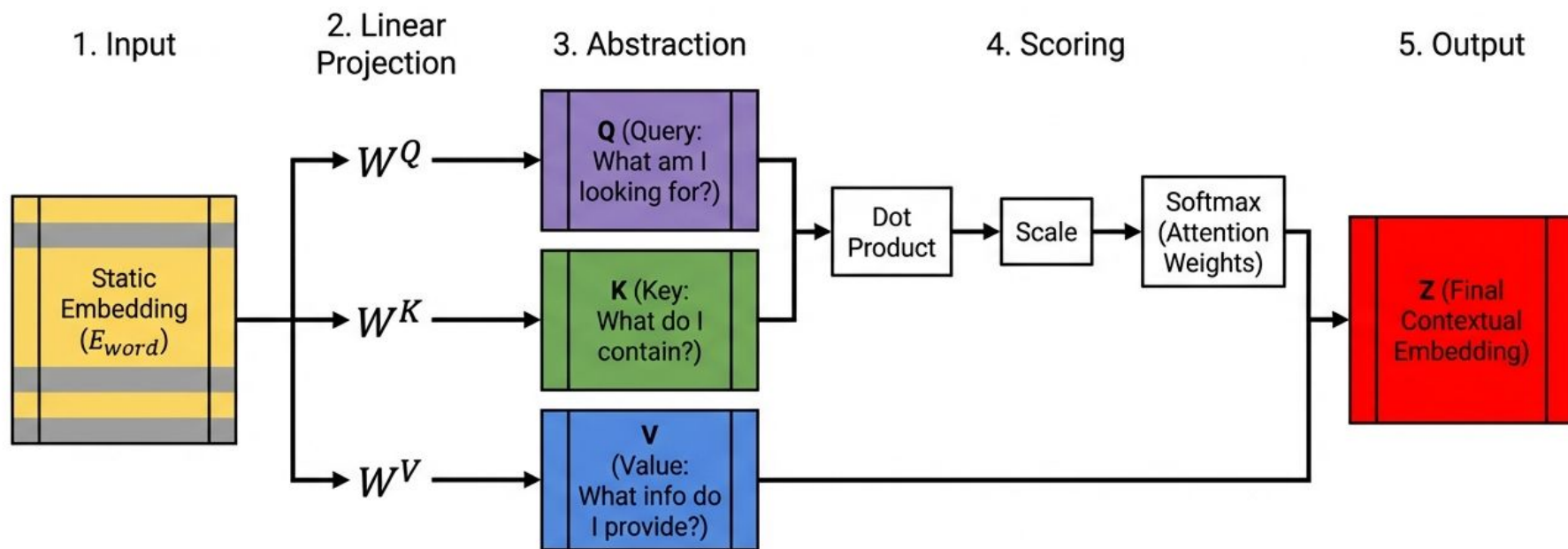


Mechanism: Dynamic vectors shaped by surrounding sequence words.

The Solution: The embedding for Bank shifts based on its neighbors.

Result: Captures true semantic meaning based on contextual usage.

The Mathematical Machinery of a Single Attention Head

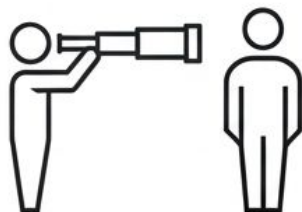


The Critical Question: What happens if a single sequence requires multiple, multiple, distinct contextual interpretations simultaneously?

The Limitation: Semantic Ambiguity

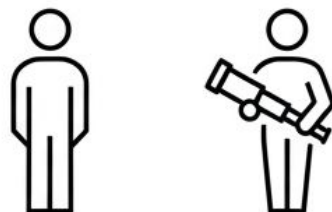
The man saw the astronomer with a telescope.

Interpretation A:
The Instrument Perspective



- The man is using the telescope to see the astronomer.
- Strong linguistic link: Man $\leftrightarrow$ Telescope

Interpretation B:
The Possession Perspective



- The man sees an astronomer, and the astronomer is holding a telescope.
- Strong linguistic link: Astronomer $\leftrightarrow$ Telescope

Natural Language is inherently ambiguous. A robust NLP model must capture all possible perspectives of a sequence before generating an output.

Why Single-Head Attention Fails at Ambiguity

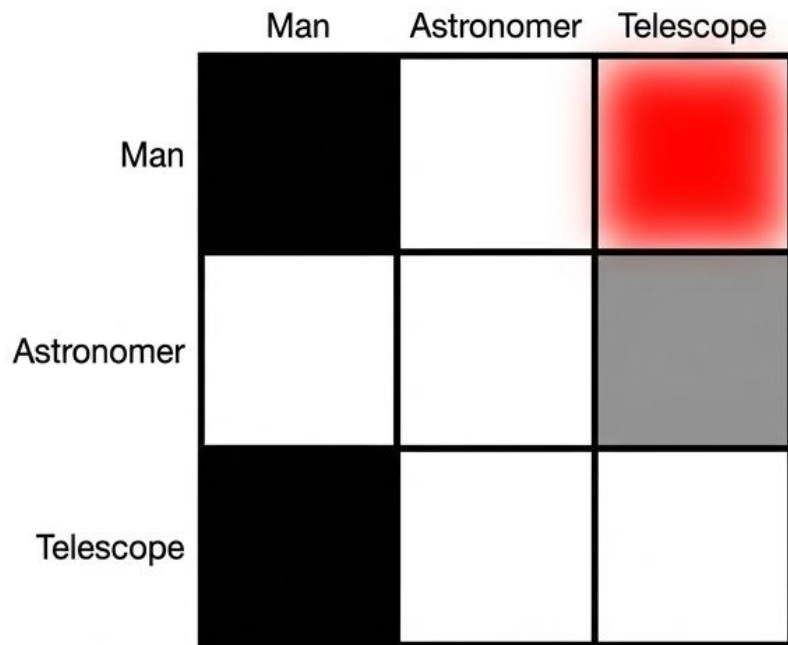
The Mathematical Bottleneck

The Single Perspective Bottleneck: A standard Self-Attention block uses only one set of W^Q , W^K , W^V matrices. During training, these weights optimize to capture only the most dominant semantic relationship.

The Trade-off: The model is forced to choose. It calculates a high similarity score for either [Man $\leftrightarrow$ Telescope] or [Astronomer $\leftrightarrow$ Telescope].

Softmax Loss: It cannot maintain both semantic interpretations simultaneously. The secondary perspective is mathematically washed out in the Softmax normalization.

Attention Heatmap

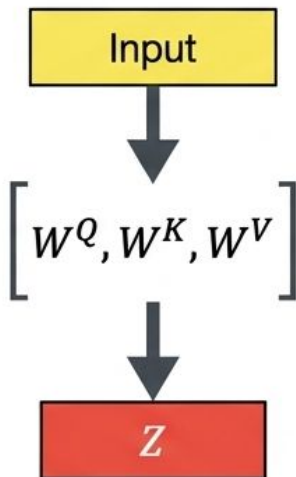


Result: Single-Head Attention = Single Perspective.

The Solution: Multi-Head Attention

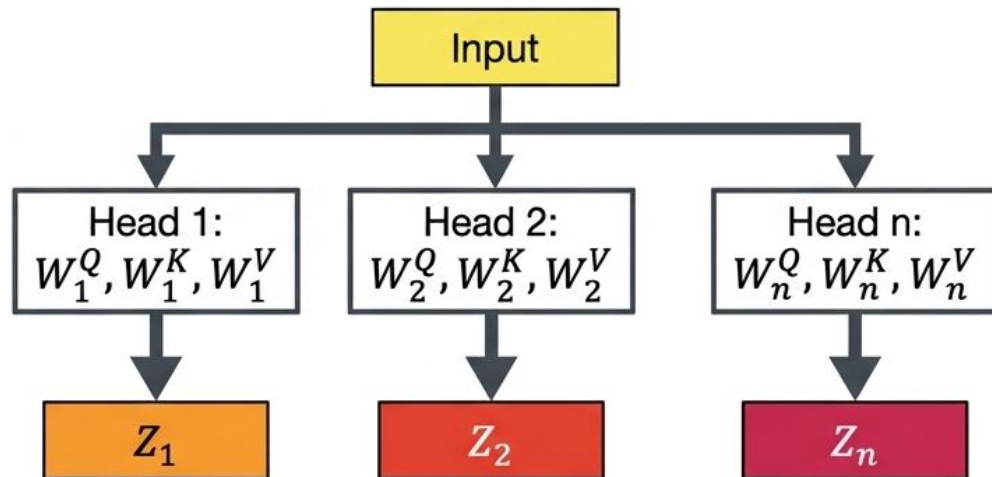
If one Self-Attention mechanism (a Head) acts as a single lens capturing one perspective, the solution is to deploy multiple lenses in parallel.

Single Head



1 Set of Weights = 1 Perspective

Multi-Head

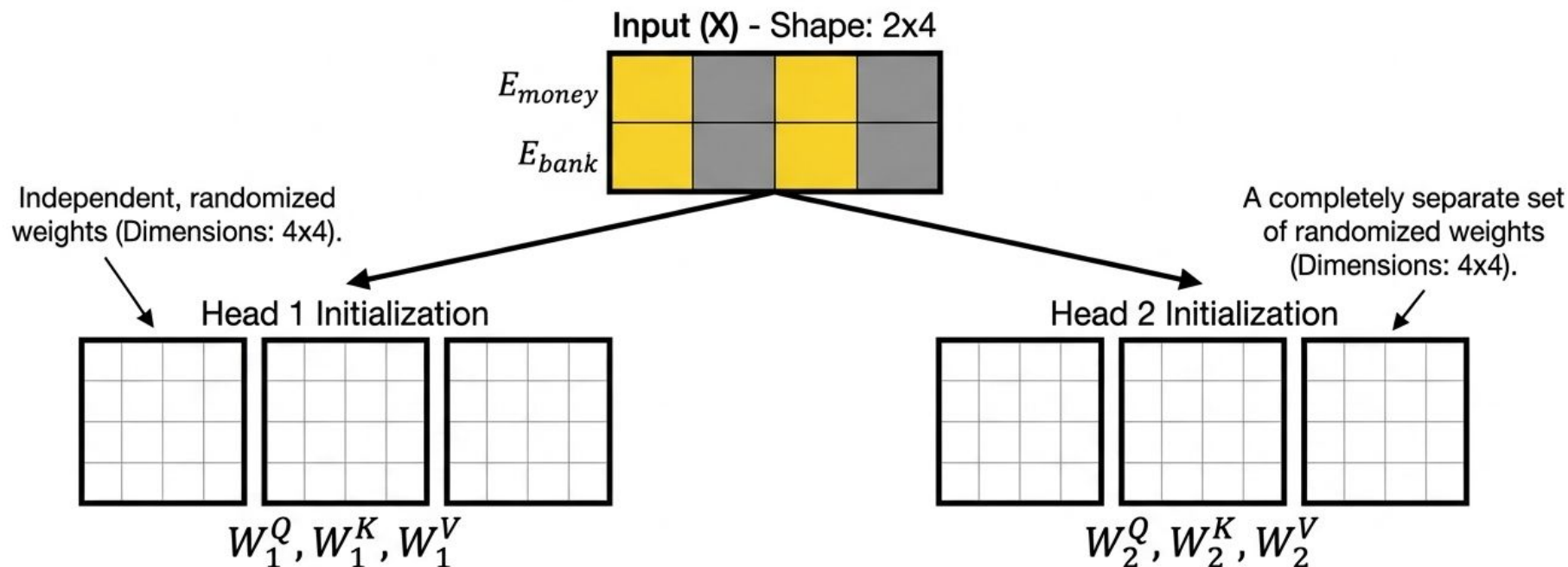


N Sets of Weights = N Perspectives

Multi-Head Attention doesn't change the underlying math of Self-Attention; it simply duplicates the machinery to allow the model to jointly attend to information from different representation subspaces.

Step 1: Expanding the Weight Matrices (Toy Example)

Let's process a 2-word sequence (Money, Bank) through two parallel Attention Heads.



Because Head 1 and Head 2 are initialized with different random weights, they will learn to project the exact same input embeddings into completely different representation spaces during training.

Step 2: Generating Parallel Q, K, V Vectors

For Head 1:

$$X * W_1^Q \rightarrow \begin{array}{|c|c|c|} \hline & Q_1 & \\ \hline & & \\ \hline \end{array} \quad (\text{Contains } Q_{money_1}, Q_{bank_1})$$

$$X * W_1^K \rightarrow \begin{array}{|c|c|c|} \hline & K_1 & \\ \hline & & \\ \hline \end{array} \quad (\text{Contains } K_{money_1}, K_{bank_1})$$

$$X * W_1^V \rightarrow \begin{array}{|c|c|c|} \hline & V_1 & \\ \hline & & \\ \hline \end{array} \quad (\text{Contains } V_{money_1}, V_{bank_1})$$

Key Insight: Every single word in the sequence now possesses TWO distinct Query vectors, TWO Key vectors, and TWO Value vectors.

For Head 2:

$$X * W_2^Q \rightarrow \begin{array}{|c|c|c|} \hline & Q_2 & \\ \hline & & \\ \hline \end{array} \quad (\text{Contains } Q_{money_2}, Q_{bank_2})$$

$$X * W_2^K \rightarrow \begin{array}{|c|c|c|} \hline & K_2 & \\ \hline & & \\ \hline \end{array} \quad (\text{Contains } K_{money_2}, K_{bank_2})$$

$$X * W_2^V \rightarrow \begin{array}{|c|c|c|} \hline & V_2 & \\ \hline & & \\ \hline \end{array} \quad (\text{Contains } V_{money_2}, V_{bank_2})$$

Step 3: Independent Self-Attention Computations

Each head now performs the standard Scaled Dot-Product Attention in complete isolation from the other.

Head 1 Computation

$$\text{Attention}(Q_1, K_1, V_1) = \text{softmax}\left(\frac{Q_1 K_1^T}{\sqrt{d_k}}\right) V_1 = Z_1$$

Z_1 (2x4 Matrix)

Perspective Captured: e.g., Financial Context.

Head 2 Computation

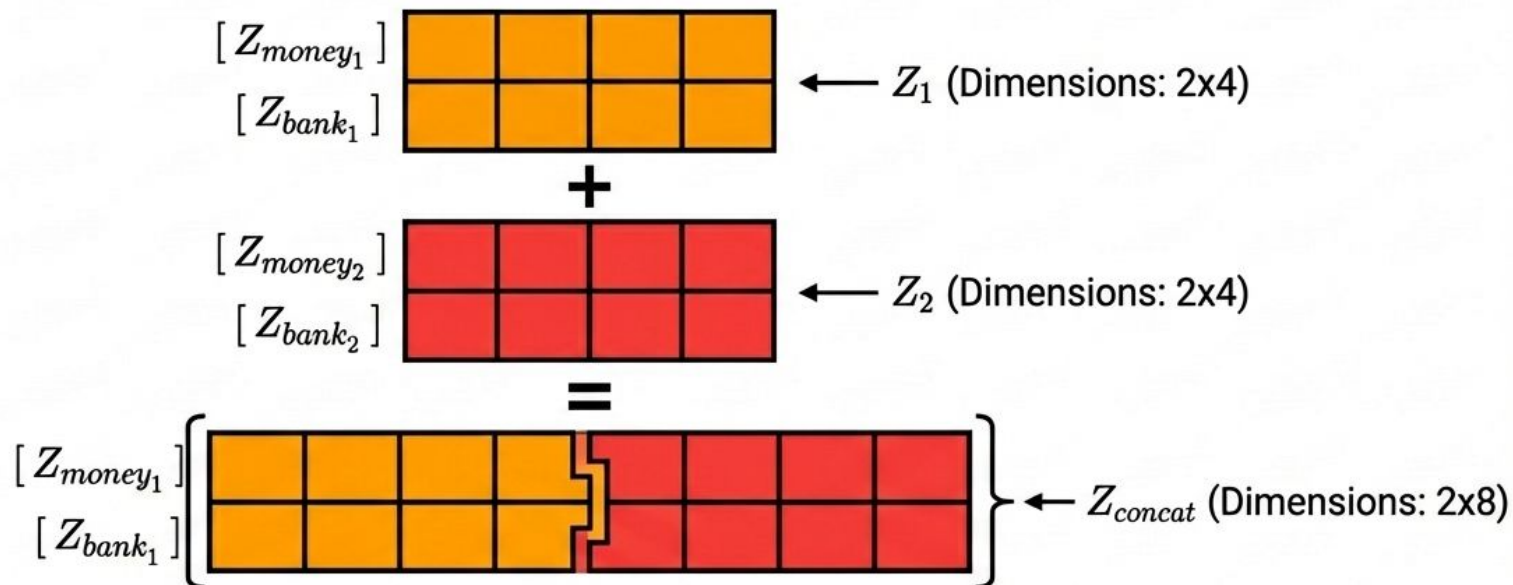
$$\text{Attention}(Q_2, K_2, V_2) = \text{softmax}\left(\frac{Q_2 K_2^T}{\sqrt{d_k}}\right) V_2 = Z_2$$

Z_2 (2x4 Matrix)

Perspective Captured: e.g., Geographical Context.

Step 4: The Concatenation Step

The downstream feed-forward neural network expects a single input matrix, not multiple isolated matrices. We must synthesize the multiple perspectives.

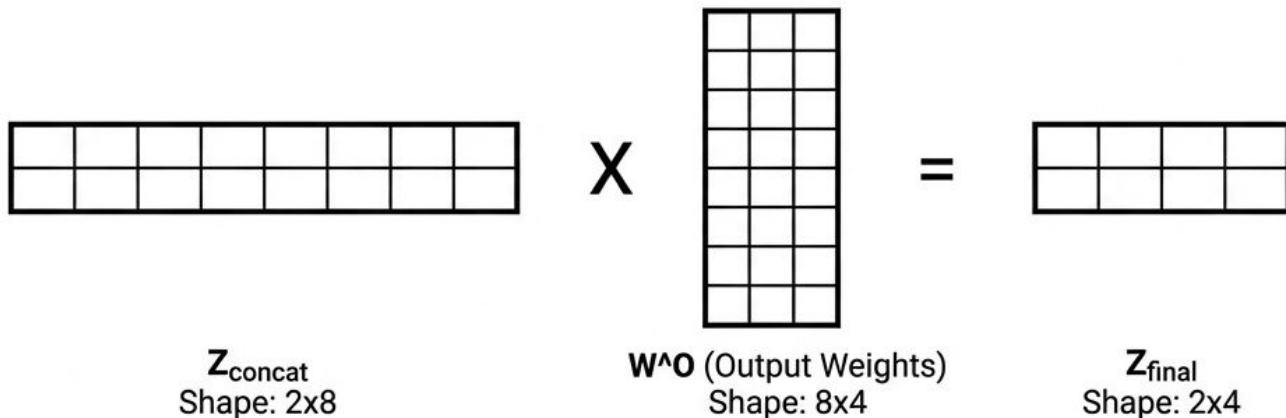


Problem: We concatenate the outputs of all heads along the feature dimension. Our input embedding was 2x4. Our concatenated matrix is now 2x8. We have altered the sequence dimensionality, breaking the continuous flow of the Transformer.

Step 5: Final Linear Projection (W^O)

To restore the original dimensionality and seamlessly blend the multiple perspectives, we pass the concatenated matrix through a **final Linear Transformation**.

$$Z_{\text{concat}} * W^O = Z_{\text{final}}$$



(Dimensionality Restored to match input!)

The Role of W^O : The W^O matrix acts as the **Perspective Mixer**. During training, it learns how to weigh and blend the isolated outputs of Head 1 and Head 2 to create the mathematically optimal, unified Contextual Embedding for each word.

Architecture Synthesis: Single-Head vs. Multi-Head

Feature	Single-Head Attention	Multi-Head Attention
Weight Matrices	1 Set (W^Q, W^K, W^V)	N Sets (W_i^Q, W_i^K, W_i^V)
Semantic Capacity	Captures 1 dominant relationship per word	Captures N distinct relationships per word
Linear Projections	Only at the input stage	At input stage + Final output stage (W^O)
Output Aggregation	None required	Requires Concatenation + Projection
Risk of Bottleneck	High (Meaning gets averaged out)	Low (Meanings isolated in specific subspaces)

Synthesis: Multi-Head Attention provides the best of both worlds by expanding representational capacity without breaking the input/output dimension expectations of the overall Transformer architecture.

Implementation: Attention Is All You Need (2017)

How was this theory actually implemented in the original Google Transformer paper?

The Hyperparameters

512

Model Dimensionality (d_{model})

(Every word embedding is a massive 512-dimensional vector)

8

Number of Heads (h)

(8 separate parallel Self-Attention modules)

The Engineering Challenge

If we run **8 independent heads**, each processing 512-dimensional vectors in parallel, the computational cost would be **8X higher** than a single-head model.

How did the authors solve this without destroying performance and speed?

The Optimization: Dimensionality Reduction

To keep computational costs identical to Single-Head Attention, the authors divided the total embedding dimensions equally among the multiple heads.

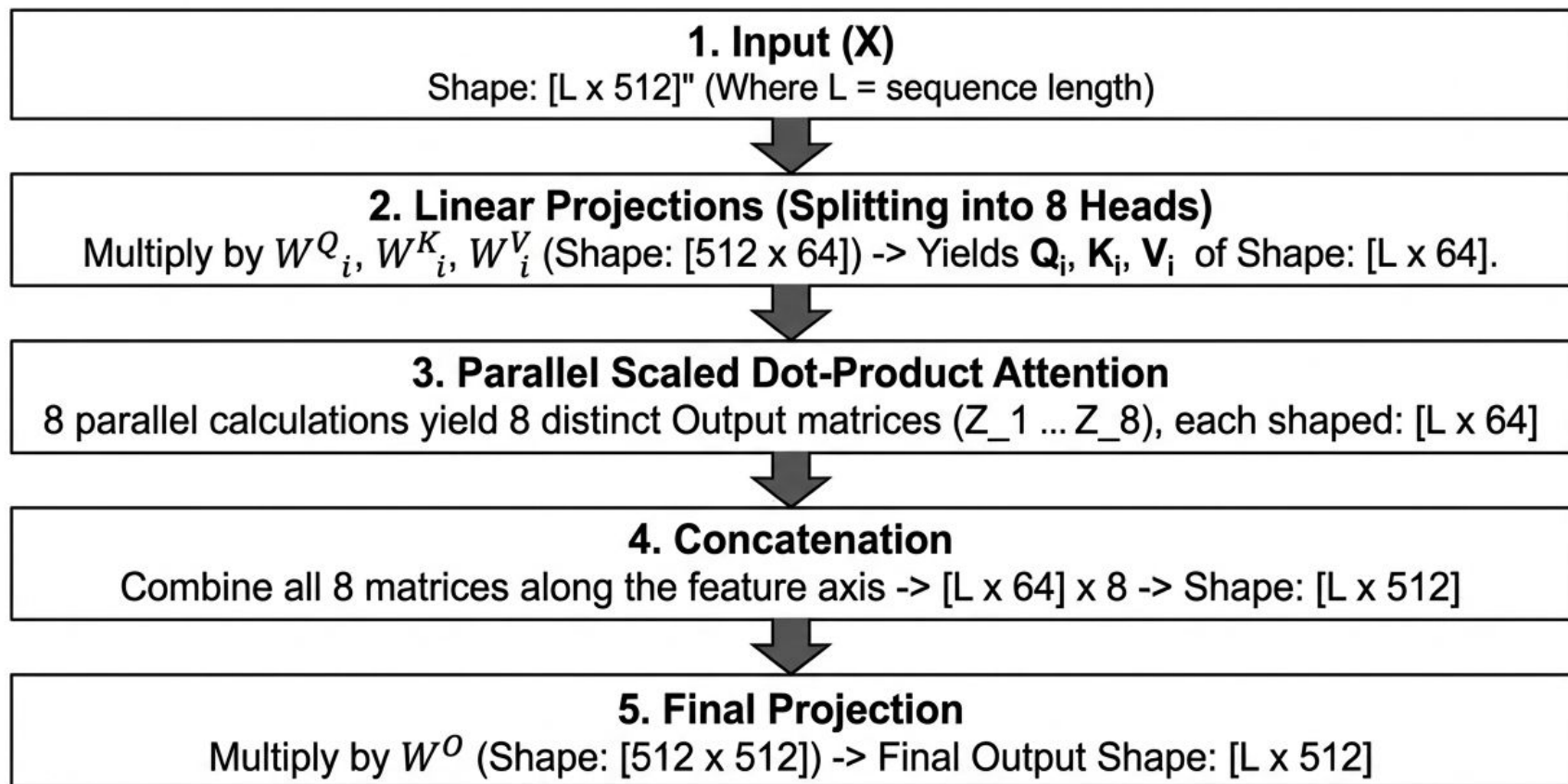
$$d_k = d_v = d_{model}/h$$
$$512/8 = 64 \text{ dimensions per head.}$$

When the 512-dimensional Input is multiplied by the matrices for Head 1, those weight matrices (W_1^Q, W_1^K, W_1^V) are specifically shaped as [512 x 64] to compress the output.

- **Single-Head Compute:** One massive operation on 512 dimensions.
- **Multi-Head Compute:** Eight lighter parallel operations on 64 dimensions.

Result: Multi-Head **Attention** captures **8 distinct perspectives** for the exact same computational cost as a single head.

The Complete Multi-Head Architecture (with Dimensions)

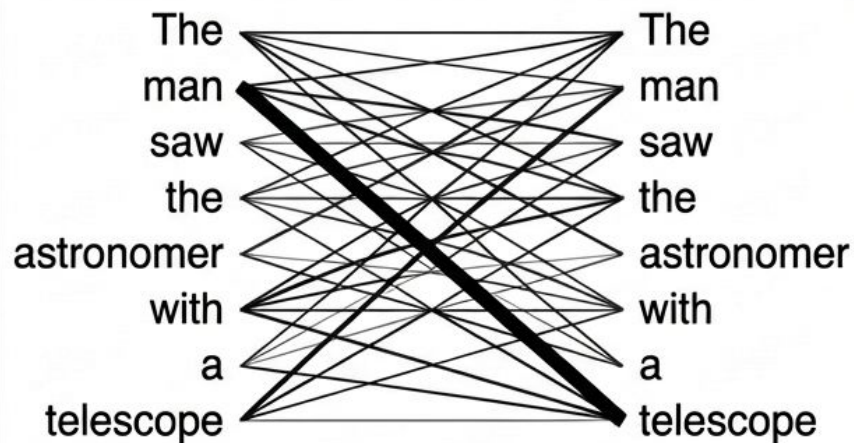


Empirical Proof: Visualizing Attention Heads

Using Google's Tensor2Tensor visualization tool, we can peer inside the layers of a trained Transformer to observe what individual heads actually learn.

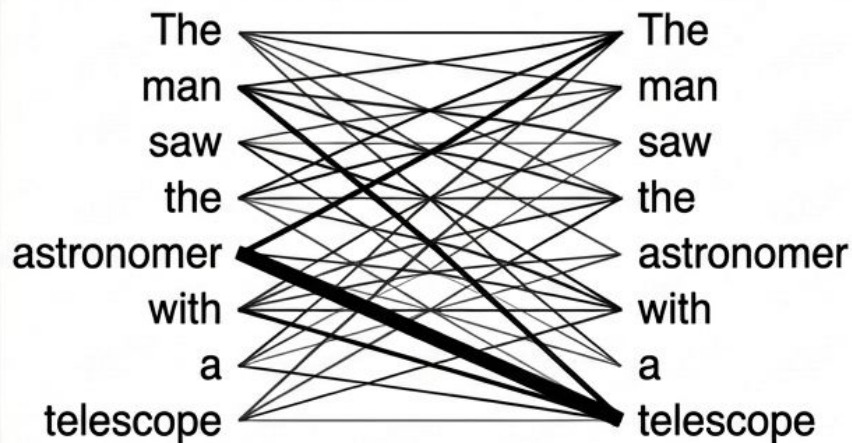
Target Sentence: The man saw the astronomer with a telescope.

Layer 0, Head 1



Interpretation learned: The man is using the telescope.

Layer 0, Head 2

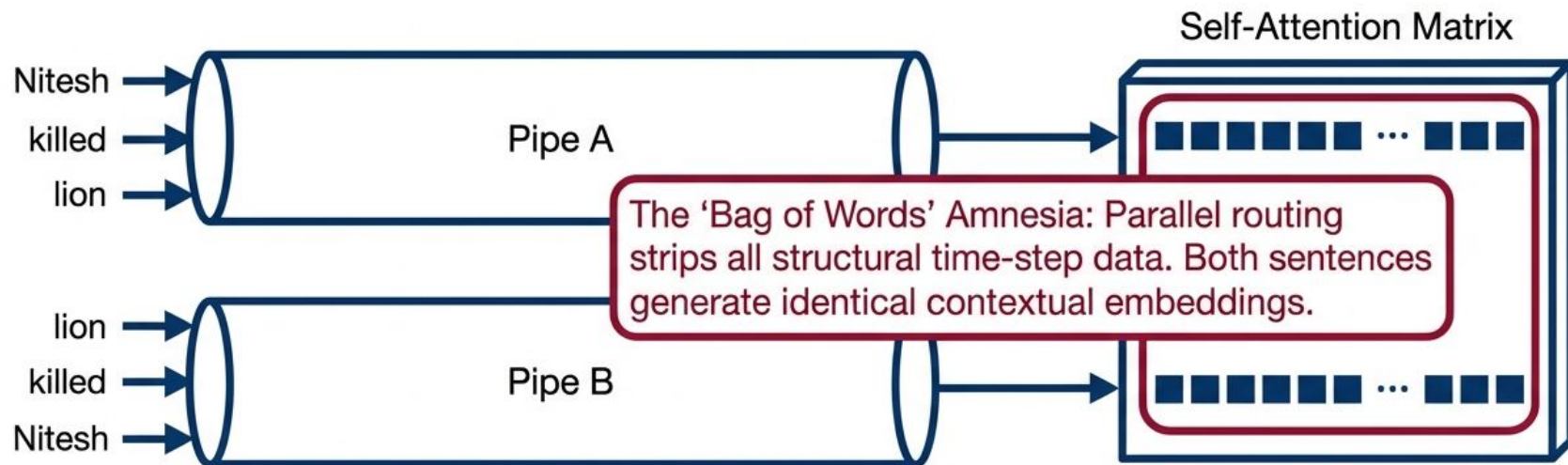


Interpretation learned: The astronomer possesses the telescope.

Conclusion: Multi-Head Attention successfully resolves linguistic ambiguity by assigning orthogonal sub-tasks to parallel heads, ensuring no critical contextual perspectives are lost in the math.

Positional Encoding

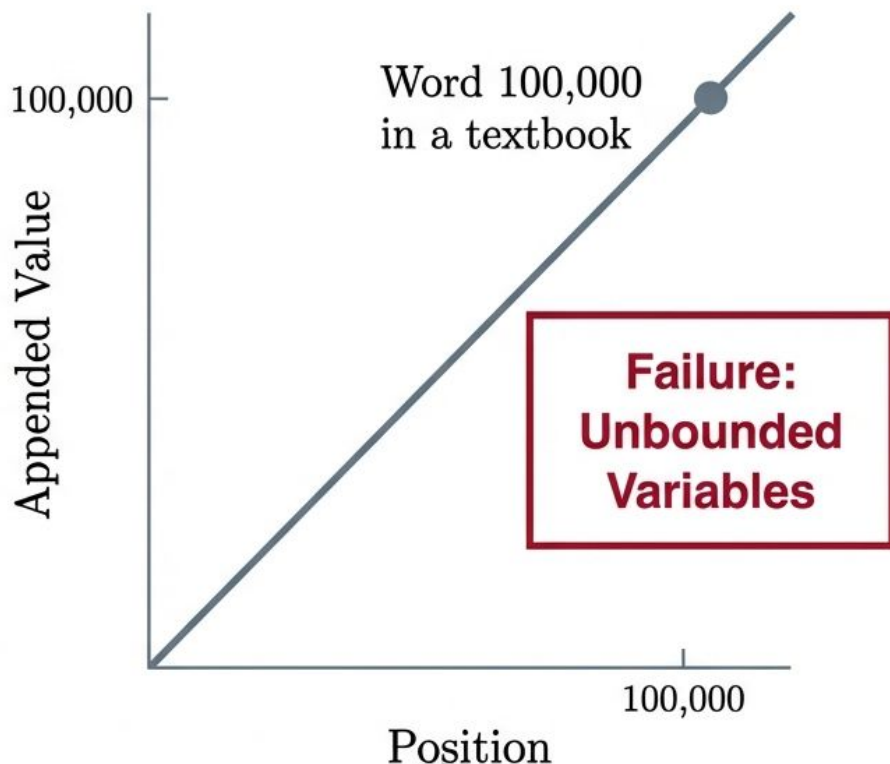
The Blind Spot of Parallel Processing



To make real-world NLP applications viable, we must artificially inject sequential order back into the embedding space before self-attention occurs.

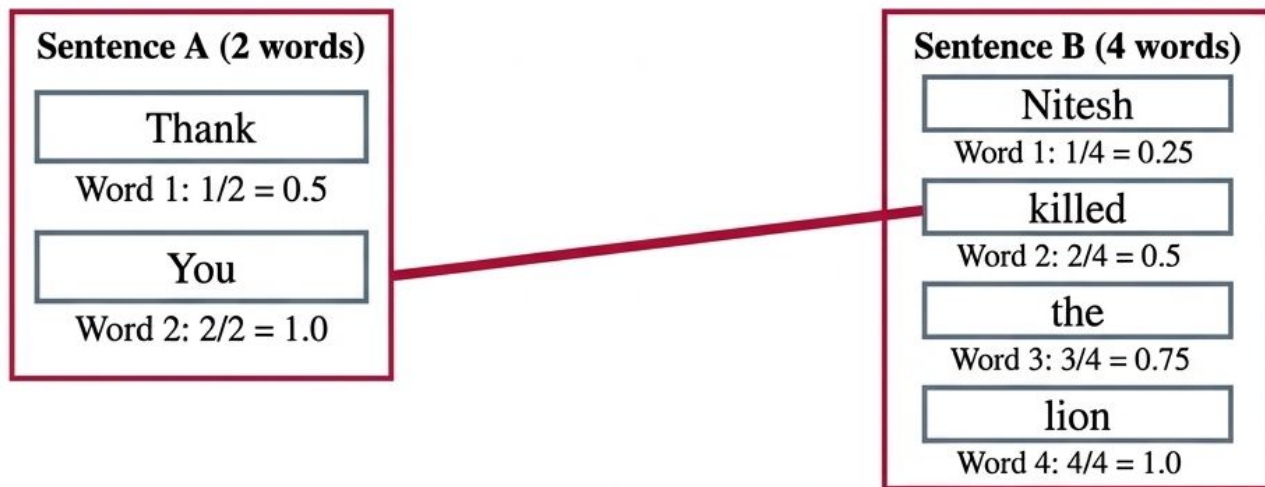
Attempt 1: Absolute Counting

Word	Embedding Vector
Nitesh	[..., 1]
killed	[..., 2]
the	[..., 3]
lion	[..., 4]



Deep neural networks rely on **Backpropagation**, which requires stable gradients (ideal range: -1 to 1). Injecting scalar values of $100,000+$ creates catastrophic numerical instability and exploding gradients during training.

Attempt 2: Normalized Counting



Failure: Semantic Inconsistency

Normalizing by maximum sequence length forces values between 0 and 1, solving gradient explosion. However, the mathematical identity of 'Position 2' is now completely relative to document length. In Sentence A, Position 2 equals 1.0. In Sentence B, Position 2 equals 0.5. The network cannot learn a stable representation for sequential placement.

The Mathematical Requirements for Positional Identity

1. Bounded & Stable

Must output values strictly within a normalized range (e.g., -1 to 1) to survive Backpropagation without causing exploding gradients.

2. Continuous & Smooth

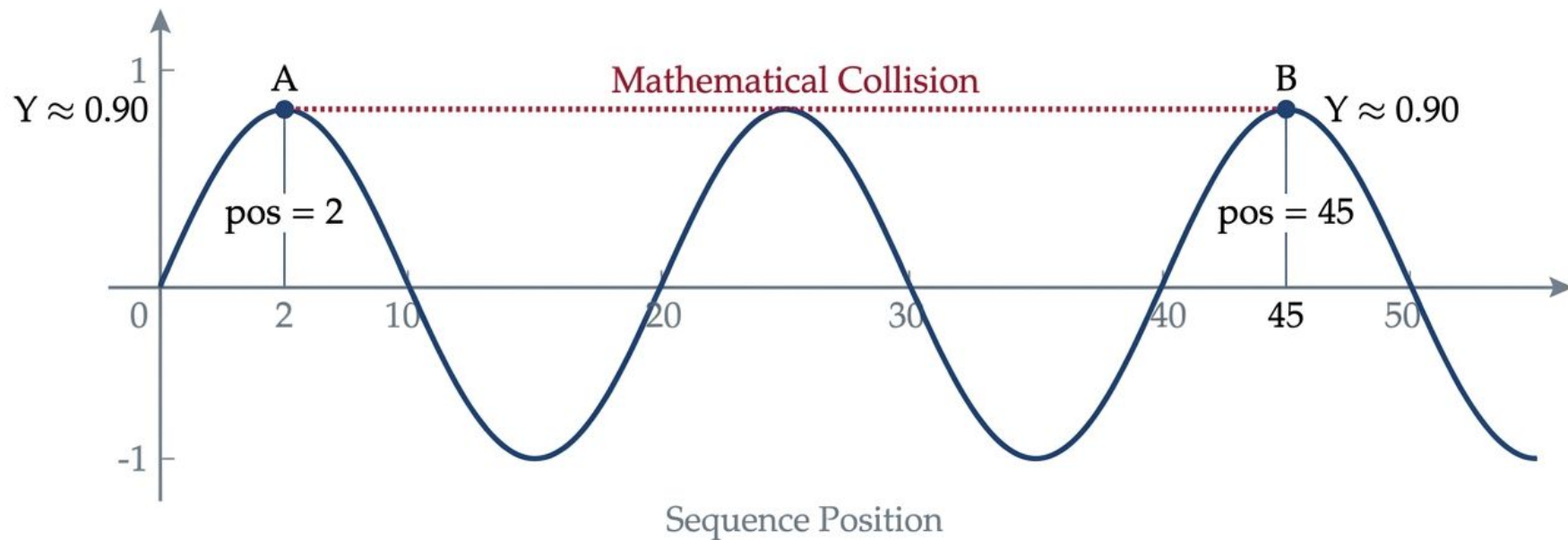
Must avoid discrete jumps (1, 2, 3) which disrupt the gradient flow. Neural networks optimize best over continuous, differentiable functions.

3. Absolute & Consistent

A specific position (e.g., pos = 4) must output the exact same unique mathematical signature regardless of whether the document is 10 words or 10,000 words long.

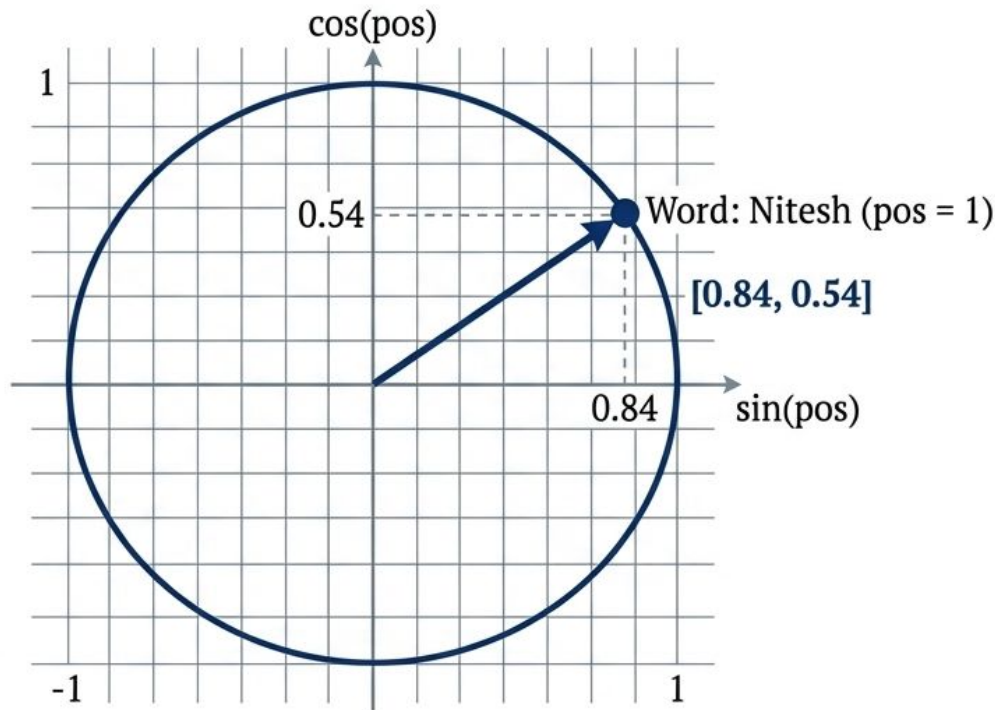

$$y = \sin(pos)$$

Attempt 3: The Periodicity Trap of a Single Wave



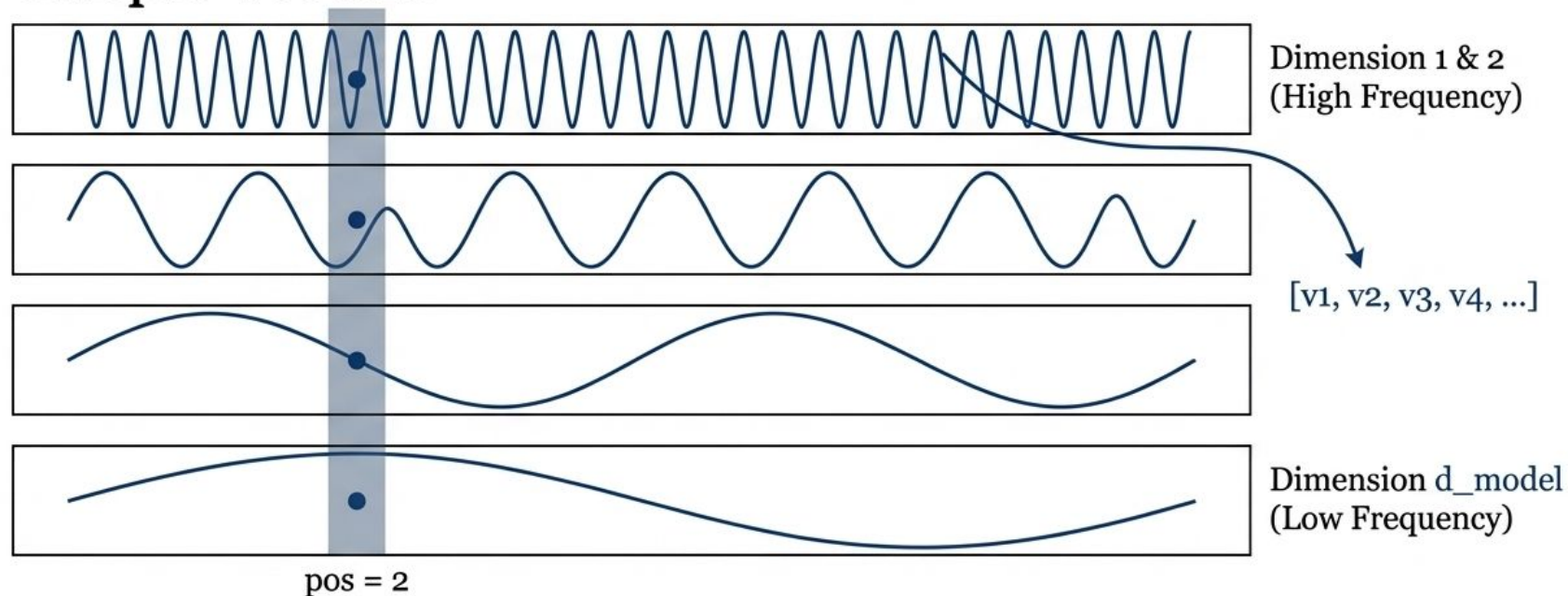
While $y = \sin(\text{pos})$ elegantly solves bounding (strictly -1 to 1) and continuity, its inherent periodicity introduces a fatal flaw. In long documents, distant words will generate the exact same positional scalar, causing the model to confuse position 2 with position 45.

Attempt 4: Multi-Dimensional Trigonometric Vectors



By evaluating the position through both a Sine and Cosine function, we upgrade our positional identity from a scalar to a 2D Vector: $[0.84, 0.54]$. This drastically reduces the probability of identical collisions, but it is still insufficient for massive 100,000-word sequences.

Attempt 5: Decreasing Frequencies for Infinite Unique Vectors



By stacking multiple Sine/Cosine pairs and progressively decreasing their frequencies, we create a deeply multi-dimensional vector. High frequencies capture precise local differences, while low frequencies capture macro sequence placement.

Architectural Integration: Addition vs. Concatenation

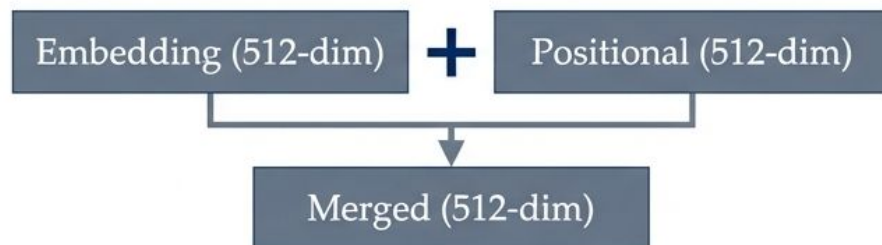
Concatenation (The Intuitive Method)



Dimensions = $2 \times d_{model}$
Parameters = Doubled
Training Time = $2 \times$



Element-wise Addition (The "Attention Is All You Need" Method)

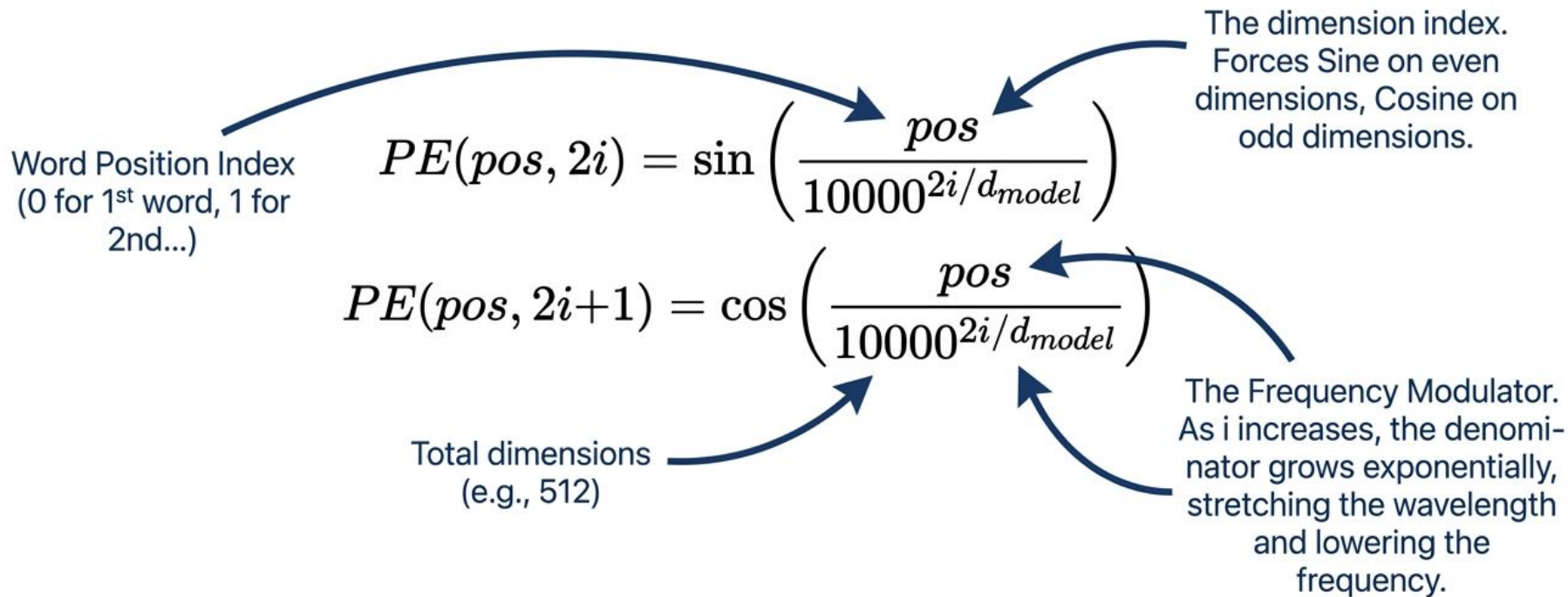


Dimensions = d_{model}
Parameters = Unchanged
Training Time = Unchanged



The Golden Rule: Because we use vector addition, the Positional Encoding dimension must exactly equal the Word Embedding dimension ($d_{model} = 512$).

Deconstructing the “Attention is All You Need” Formula



Execution Trace: Calculating Position 0 ('River')

Word: 'River' | pos = 0 | d_model = 6 (scaled down)

Dim 0 (Even):	$\sin(0 / 10000^0)$	→	$\sin(0)$	→	0
Dim 1 (Odd):	$\cos(0 / 10000^0)$	→	$\cos(0)$	→	1
Dim 2 (Even):	$\sin(0 / 10000^{2/6})$	→	$\sin(0)$	→	0
Dim 3 (Odd):	$\cos(0 / 10000^{2/6})$	→	$\cos(0)$	→	1
Dim 4 (Even):	$\sin(0 / 10000^{4/6})$	→	$\sin(0)$	→	0
Dim 5 (Odd):	$\cos(0 / 10000^{4/6})$	→	$\cos(0)$	→	1

PE_River = [0, 1, 0, 1, 0, 1]

Because the numerator is 0, the frequency denominator is mathematically irrelevant. Position 0 will always yield an alternating boolean-style sequence.

Execution Trace: Calculating Position 1 ('Bank')

Word: 'Bank' | pos = 1 | d_model = 6

Dim 0: $\sin(1 / 1) \rightarrow \sin(1) \rightarrow 0.84$

Dim 1: $\cos(1 / 1) \rightarrow \cos(1) \rightarrow 0.54$

Dim 2: $\sin(1 / 21.5) \rightarrow \sin(0.04) \rightarrow 0.04$ (Notice denominator growth)

Dim 3: $\cos(1 / 21.5) \rightarrow \cos(0.04) \rightarrow 0.99$

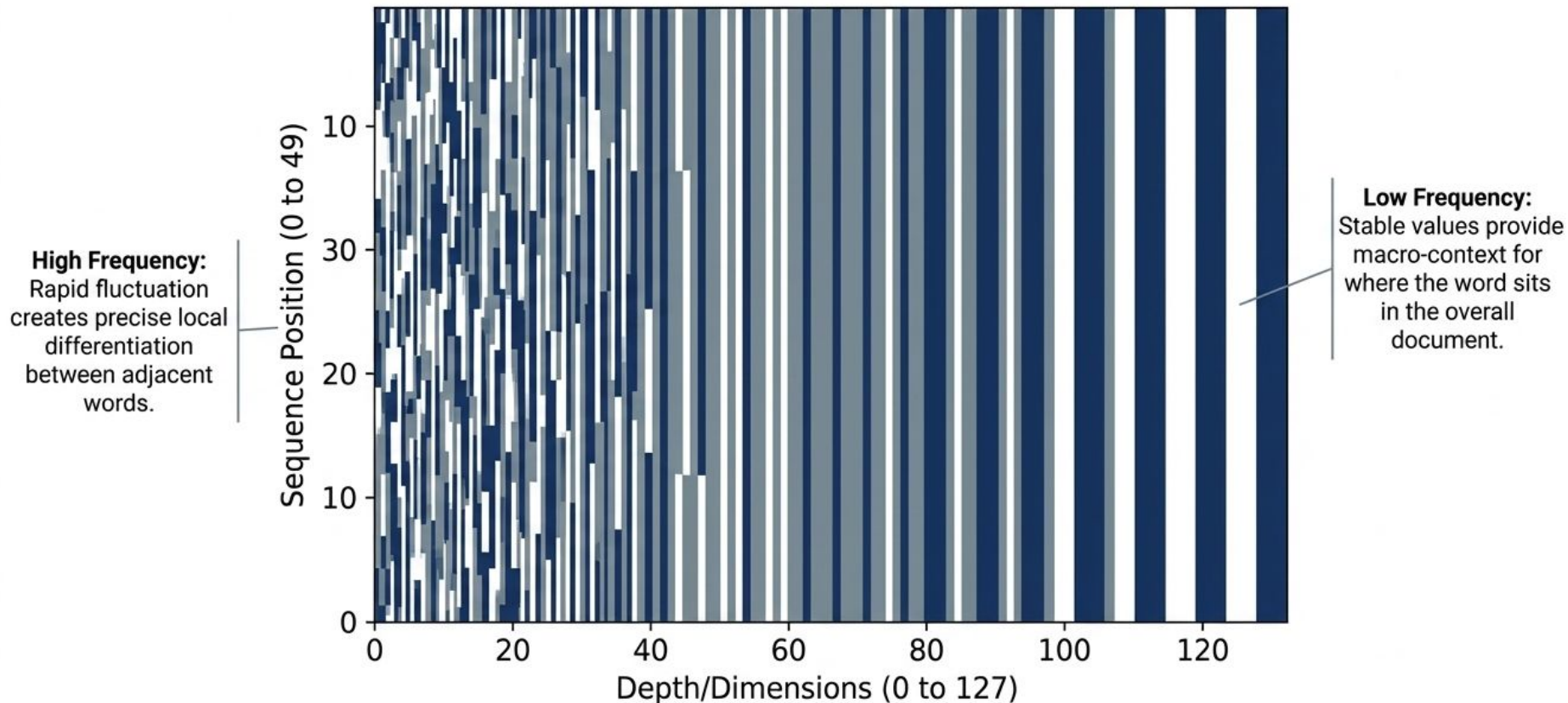
Dim 4: $\sin(1 / 464.1) \rightarrow \sin(0.002) \rightarrow 0.002$ (Extreme denominator growth)

Dim 5: $\cos(1 / 464.1) \rightarrow \cos(0.002) \rightarrow 0.999$

$$PE_{Bank} = [0.84, 0.54, 0.04, 0.99, 0.002, 0.999]$$

As the dimension i increases, the denominator grows massively. This forces the internal value toward 0, making high-dimension Sine evaluate to 0 and Cosine evaluate to 1.

Visualizing the Positional Encoding Matrix



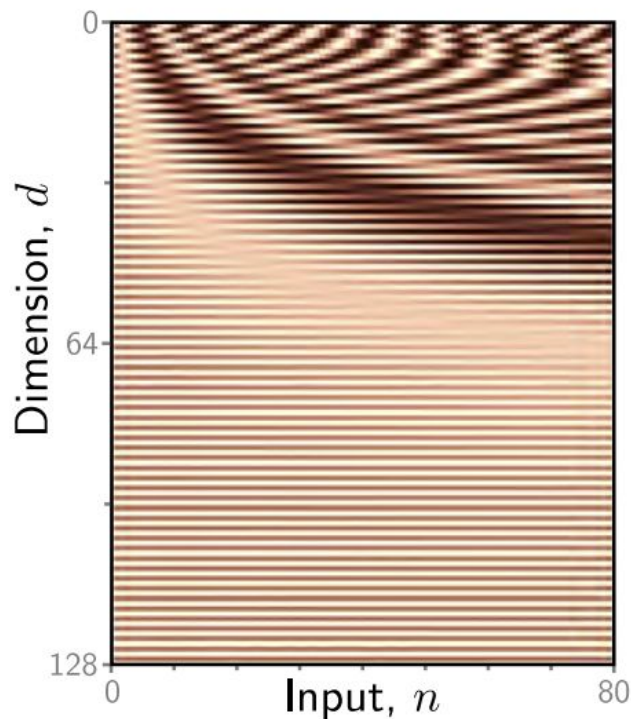
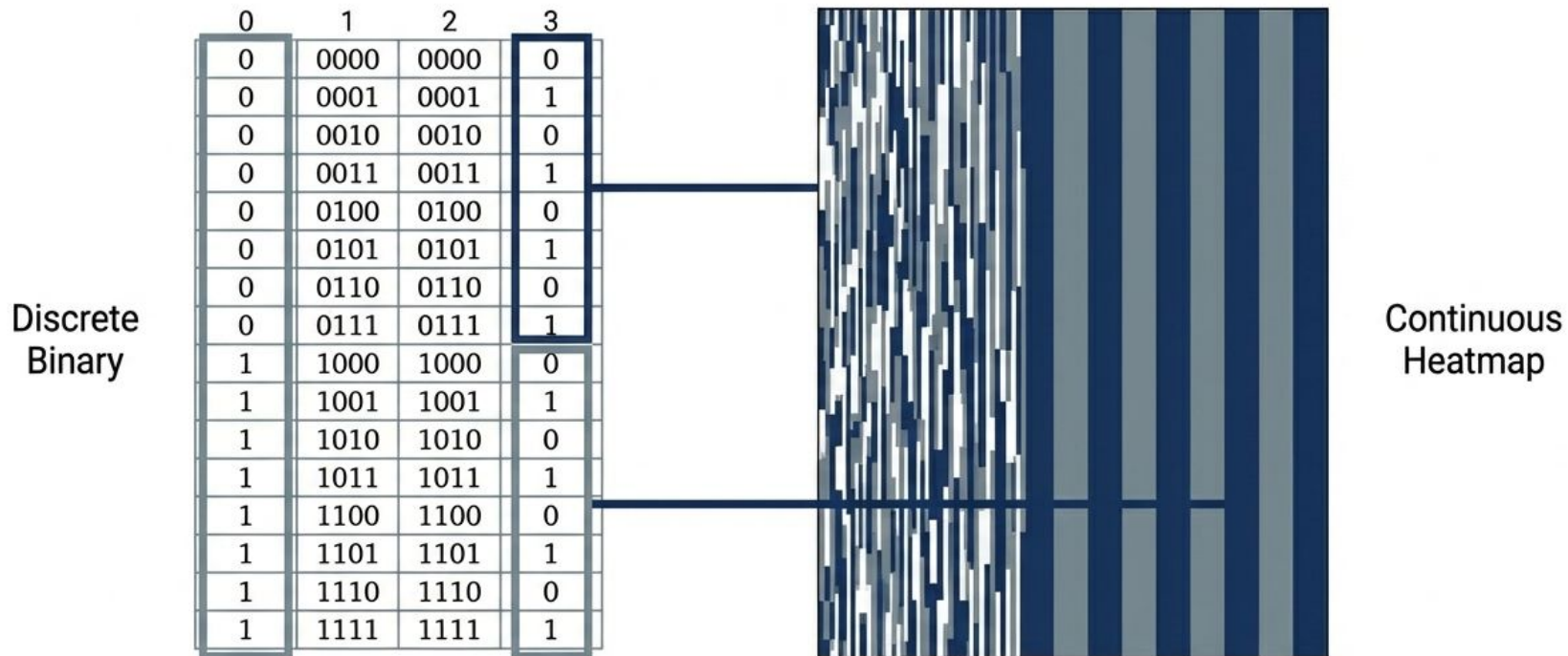


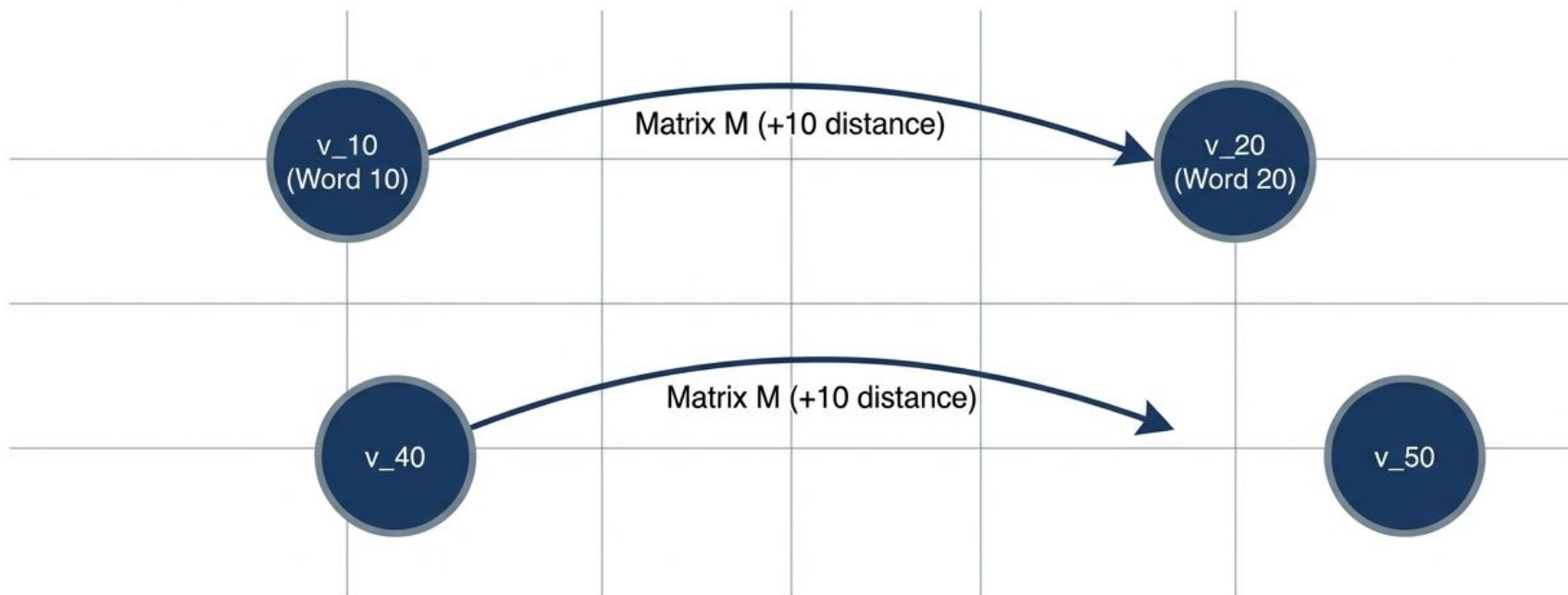
Figure 12.5 Positional encodings. The self-attention architecture is equivariant to permutations of the inputs. To ensure that inputs at different positions are treated differently, a positional encoding matrix $\mathbf{\Pi}$ can be added to the data matrix. Each column is different, so the positions can be distinguished. Here, the position encodings use a predefined procedural sinusoidal pattern (which can be extended to larger values of N if necessary). However, in other cases, they are learned.

Conceptual Analogy: Continuous Binary Encoding



The brilliant intuition behind the math: it is simply binary counting translated into the domain of continuous floating-point numbers. It replicates the bit-flipping pattern without breaking neural network continuity requirements.

The Hidden Superpower: Relative Position Awareness



While our system assigns absolute positions (1, 2, 3), language requires relative understanding (e.g., “**how far is lion from killed?**”). Because of the trigonometric properties of **Sine** and **Cosine**, the network can easily learn to calculate the distance between any two words using a single linear transformation matrix.

The Proof of Relative Transformation

$$\mathbf{PE}_{(\text{pos}+k)} = \mathbf{M} \cdot \mathbf{PE}_{\text{pos}}$$

- PE_{pos} : The known vector of the current word.
- k : The physical distance offset (e.g., +5 words).
- M : A learned linear transformation weight matrix.

Driven by the angle addition formulas:

$$\sin(a + b) = \sin(a)\cos(b) + \cos(a)\sin(b)$$

$$\cos(a + b) = \cos(a)\cos(b) - \sin(a)\sin(b)$$

Because $\text{pos}+k$ can be expanded using basic trigonometric addition, the distance k can be isolated into a fixed matrix M . The neural network does not need to memorize infinite distances; it simply learns the weight matrix M to perfectly understand relative positioning across the entire document.